

Generative AI using Large Language Models (LLMs)

Lecture 3 | 02/06/25

Instructor: Chen Wang
Fri 4:10 pm - 6:40 pm | 517 Hamilton Hall

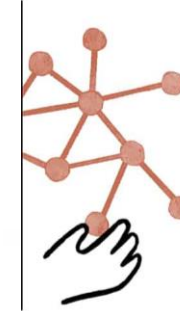


What are the LLM Products you have used?



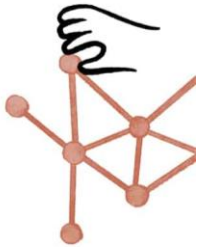
ChatGPT

Gemini



Meet
Claude

A next-generation AI assistant for your tasks, no matter the scale





Prompts and completions

Prompt

What is Large Language Model?

Context window: 2k – 100K

Model

LLM

Completion

What is Large Language Model?
A **Large Language Model (LLM)** is a type of artificial intelligence (AI) model designed to understand, generate, and manipulate human language on a large scale. These models are typically built using deep learning techniques, particularly neural networks, and are trained on massive amounts of text data to learn the statistical patterns and structures of language.



What are the LLM use cases you have seen?

- Other Use Cases
 - Translate
 - Summarize
 - Proof-read and correct
 - Explain words
 - Create article/email
 - Make restaurant suggestions
 - Chat with users
 - Do Math Calculations
 - Answer Questions on many subjects
 - ...



Could you help me draft an email saluting my PhD advisor Prof. Kim for ThanksGiving?



Certainly! Here's a di



Subje

Dear F

I hope

As Th

guida

deeply

Wishir

Thank

Best r

[Your Name]



Free Chrome extension to generate entire emails and messages using ChatGPT AI. All sites are supported and enhanced support for Gmail.



70,000+ happy users who don't like writing emails

- ✓ Highest quality responses that outcompete all other tools in the market
- ✓ Works in all Chromium-based browsers, such as Chrome, Brave, and Edge



SMARTLING

lation



Google Translate

translate "塞翁失马，安知祸福" in English.

the phrase 塞翁失马，安知祸福 can be translated



amazon Translate

e fable a
g about



Falcon LLM

t who

that what



GitHub Copilot



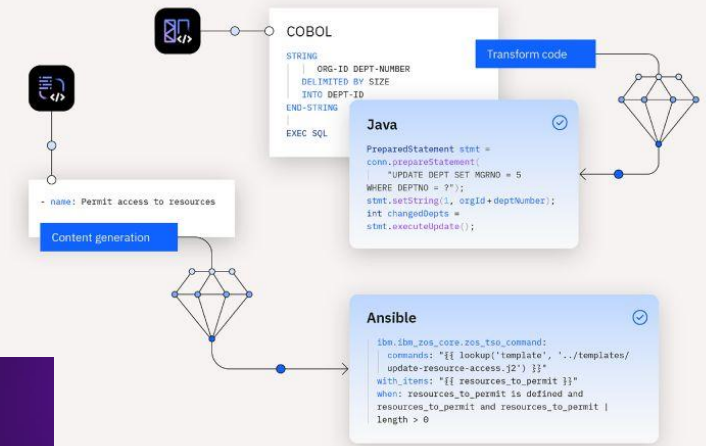
Write Flask code response with 1 request.

a simple Flask API that returns a JSON response via a GET request:

```
python  
  
from flask import Flask, jsonify
```



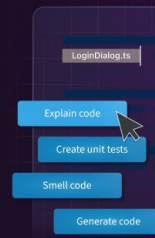
watsonx Code Assistant



Amazon Q Developer



Meet Cody,
the most powerful
& accurate AI coding
assistant



```
app.run(debug=True)
```



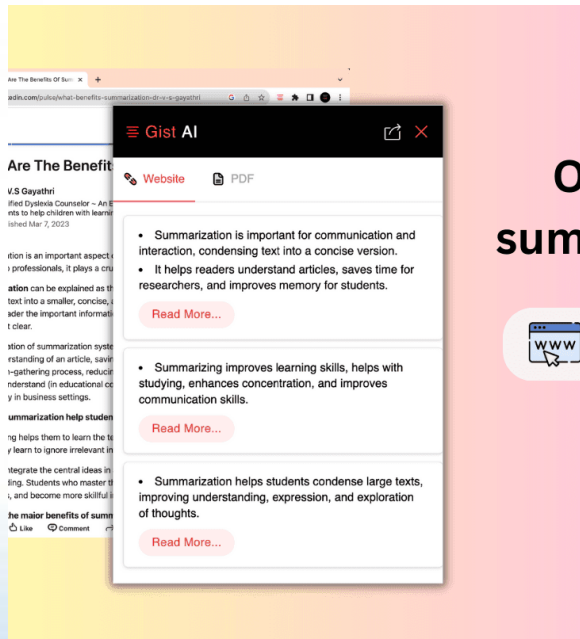
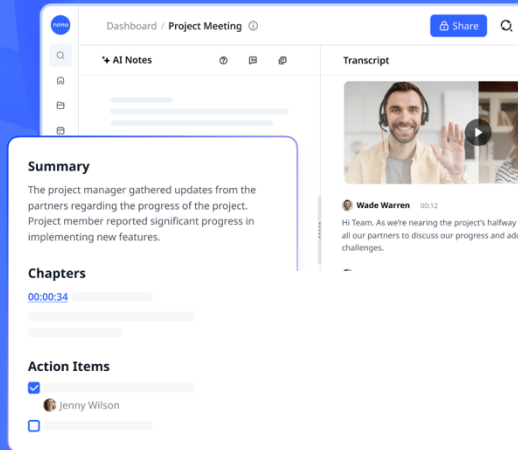
AI Assistant For Software Developers



CURSOR AI

@indai

Boost productivity with the leading AI notetaker



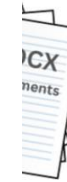
o
sum



upword*

Summarize Documents in Seconds

Easily get a Summary of DOCX files as large as 500 pages or 150K words.



Select Document

Medium Summary ▾

Get Summary

Select DOCX, PDF File & Choose Summary Size

Read or Download Summary Easily

1 with execution environments.

Is on SWE-bench, including proprietary models and their forming one (Claude 2), can resolve only a small fraction

lled SWE-bench-train, which consists of 19,000 non-dels based on CodeLlama, which are specialized for

The benchmark aims to push the boundaries of LM capabilities in practical and autonomous software engineering, providing a realistic and sustainable evaluation framework that can be continuously updated with new task instances.



krisp

AI Meeting Assistant

- ✓ Free unlimited transcriptions
- ✓ Meeting notes and summaries
- ✓ Meeting audio recording

[Learn more about Krisp AI Meeting Assistant](#)

Note Taker

Emily 1:23

Sarah 1:45

Yes 1:55

Emily 2:05

Useful meeting, engaging platform. We've used it a week. Our points featured

That's great news, Emily! User use and credibility. We have analyzed target audience in responding well included in our points

Wonderful to hear, Emily and Sara the upcoming email marketing can based on their preferences to gain with yield higher conversion rates

Summary

Targets

Emily reported a 20% engagement boost in their social media campaign, emphasizing the success of user testimonials. Sarah highlighted positive feedback and interactive element effectiveness.

Progress

Mark discussed personalization strategies in the upcoming email campaign and ongoing trend research.

granola

The AI Notepad for people in back-to-back meetings

My notes + live transcript

Intro call: AllFound

3:30pm Mike +2

100, growingg

use tuesday.ai, v manual,

180

"a priority for q2"

Granola Enhanced

Intro call: AllFound

3:30pm Mike +2

AllFound Overview

- 100 employees, adding 20 more next quarter
- Office in San Francisco and Austin

Current Provider (Tuesday.ai)

- Data input is too manual
- Too complex for non-technical team members
- \$180 per employee per year

Their Requirements

- It's "a priority for Q2" to find a better employee engagement tool
- Need secure information sharing capabilities

Circleback

Automated meeting notes and follow-ups powered by AI.

Circleback.

Never take meeting notes again.

All of your meeting notes and action items you can trust.

Try for free

See how it works

Privacy Login

Meeting Summarization Tool



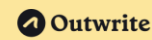


correct

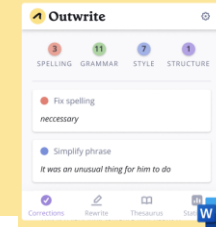


ProWritingAid

Can you proof-read and correct this sentence?
the market to buy some fruits and vegetables.'



Outwrite
for Word



Hemingway
Editor



its and vege

Ginger
Grammar
Checker





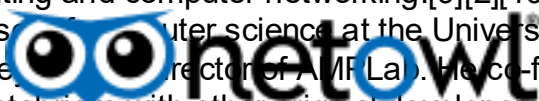
E

action

COGNIGY

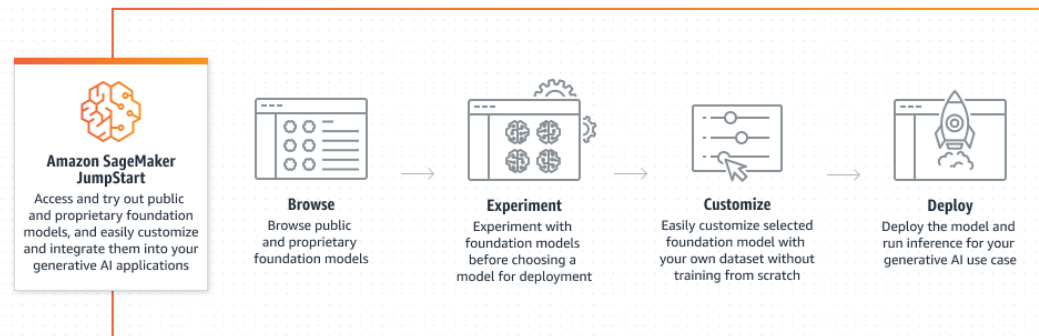
Extra
including names of people, organizations, locations, dates, and any other relevant information. For each entity, provide its type (e.g., person, organization, location, date) and the text associated with it. Output the results in a list of JSON objects, where each object contains the entity and its type.

Ion Stoica (born 1964 or 1965) is a Romanian–American computer scientist specializing in distributed systems, cloud computing and computer networking.[9][2][10][11] He is a professor of computer science at the University of California, Berkeley, and director of AMF Lab. He co-founded Conviva and Databricks with other original developers of Apache Spark.[6][12] As of April 2022, Forbes ranked him and Matei Zaharia as the 3rd-richest people in Romania with a net worth of \$1.6 billion.[13]



SINTELIX

[

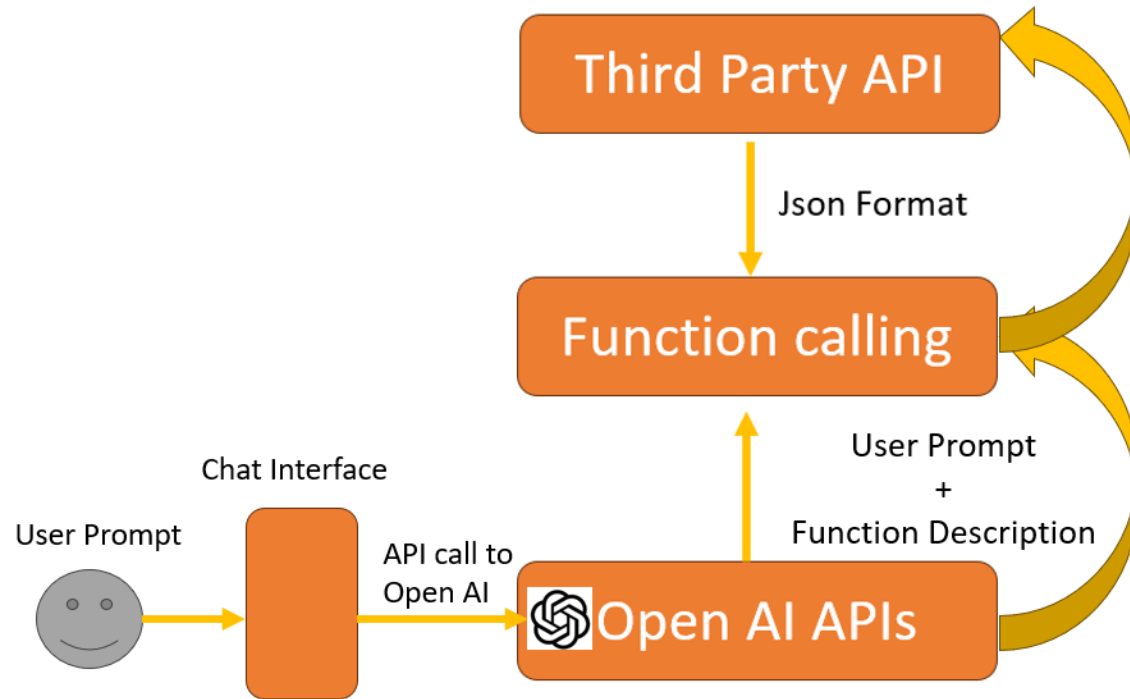


**Your training and inference data is not shared by nor used by AWS or foundation model providers*

```
{
  "type": "occupation",
  "text": "computer scientist"
},
{
  "entity": "distributed system",
  "field": "distributed system:"
}
```

: "University of California",
"organization",
University of California





tus of Ch
at it is cu
r real-tin

boomi



Power Automate





Math Calculation

Let's solve this step-by-step:

1) First, let's set up the multiplication:

$$\begin{array}{r} 237 \\ \times 56 \\ \hline \end{array}$$

2) We'll multiply 237 by 6 (the ones digit of 56):

$$237 \times 6 = 1422$$

3) Now we'll multiply 237 by 50 (the tens digit of 56):

$$237 \times 50 = 11850$$

4) Now we add these two results:

$$\begin{array}{r} 1422 \\ + 11850 \\ \hline 13272 \end{array}$$

Therefore, $237 \times 56 = 13,272$.

Would you like me to explain any part of this calculation in more detail?



To multiply 237 by 56, you can follow these steps:

Step 1: Break down the numbers

- Write the numbers in a column format:

$$\begin{array}{r} 237 \\ \times 56 \\ \hline \end{array}$$

Step 2: Multiply 237 by 6 (the ones digit of 56)

- $(237 \times 6 = 1422)$

Step 3: Multiply 237 by 5 (the tens digit of 56), but remember it's actually 50

- $(237 \times 50 = 11850)$

- You can calculate it as:

- $(237 \times 5 = 1185)$

- Then, multiply by 10 to get (11850)

Step 4: Add the results of the two multiplications

- Align the results according to their place value:

$$\begin{array}{r} 1422 \\ +11850 \\ \hline 13332 \end{array}$$

Final Result

The product of 237 and 56 is **13,332**.





Prompting and Prompt Engineering

Prompt

What is Large Language Model?

Context window: 2k – 100K

Model

LLM

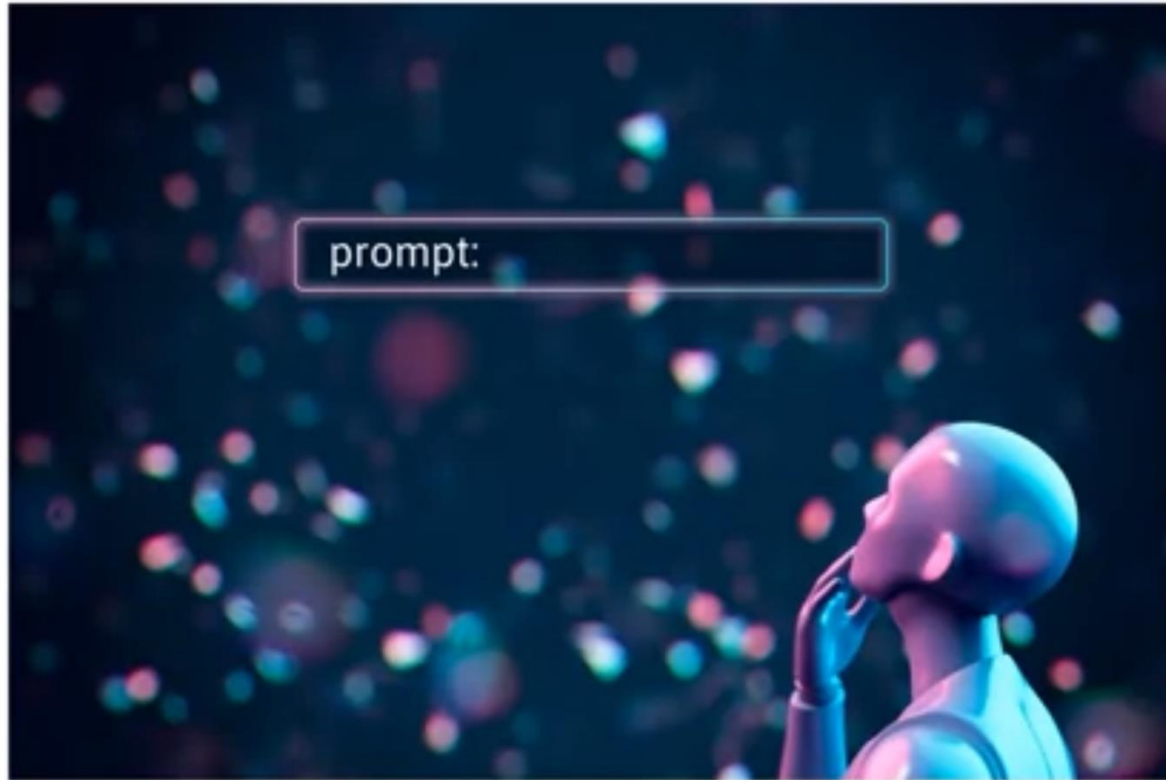
Completion

What is Large Language Model?
A **Large Language Model (LLM)** is a type of artificial intelligence (AI) model designed to understand, generate, and manipulate human language on a large scale. These models are typically built using deep learning techniques, particularly neural networks, and are trained on massive amounts of text data to learn the statistical patterns and structures of language.

You may have to revise the language in your prompt or the way that it's written several times to get the model to behave in the way that you want.

This work to develop and improve the prompt is known as **prompt engineering**.

What is a prompt?



A prompt is an input that produces the desired output.

Example:

- Write a small paragraph describing your favorite holiday destination.
- Write HTML code to generate a dropdown selection of cities within an online form.

What is a prompt?



Prompts can also be a series of instructions.

Example:

- Write a short story about a scientist studying life on Mars.
- What were some of the challenges he faced during his research?

What is a prompt?





Correct and incorrect prompts

Rich man's story
from a small
town, his
struggles, and
achievements.



Write a short story
about the struggles
and achievements
of a farmer who
became a rich and
influential
businessman in 10
years.

Context

Proper structure

Comprehensible

Correct and incorrect prompts



Correct and incorrect prompts

Sunset image
between
mountains.



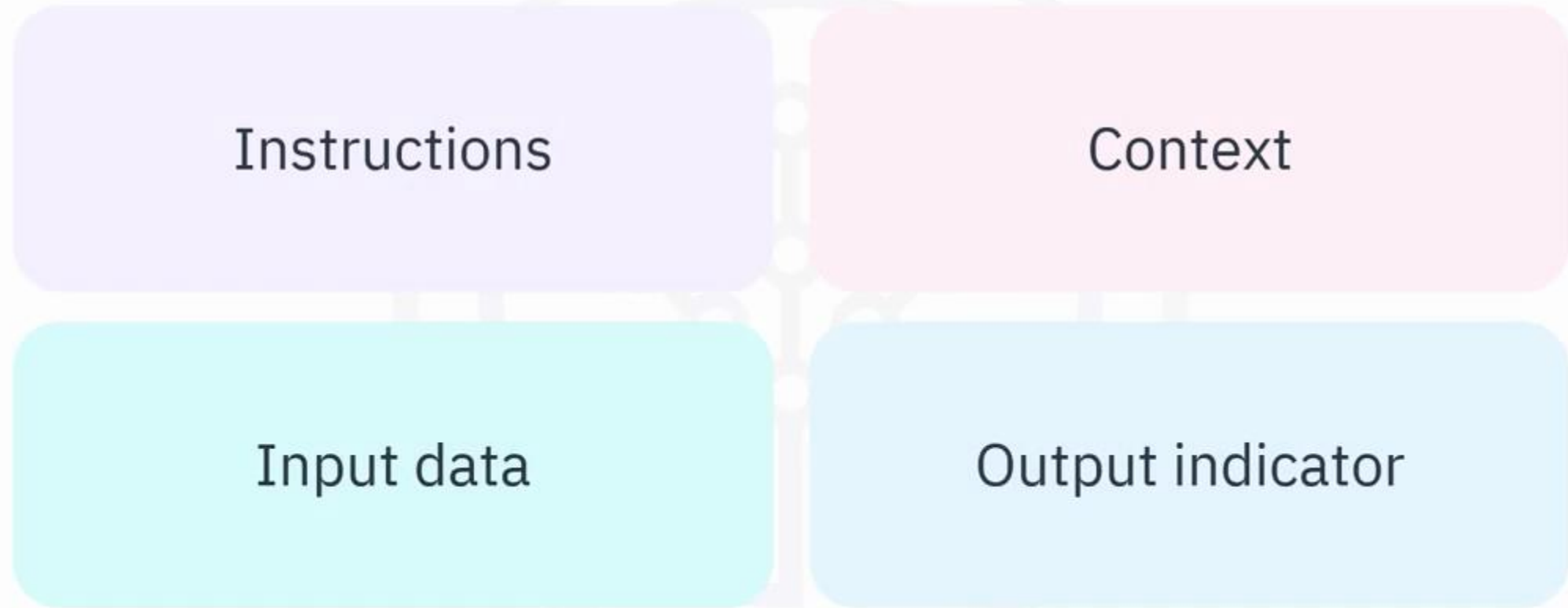
Too brief

Lacks detailed
outline



Generate an image
depicting a calm
sunset above a river
valley that rests
amidst mountains.

Building blocks of a well-constructed prompt



Building blocks of a well-constructed prompt



Instructions: Give distinct guidelines regarding the task

Example:

Write an essay in 600 words analyzing the effects of global warming on marine life.

Building blocks of a well-constructed prompt

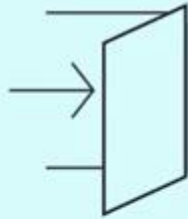


Context: Provides a framework for generating relevant content

Example:

In recent decades, global warming has undergone significant shifts, leading to rising sea levels, increased storm intensity, and changing weather patterns. These changes have had a severe impact on marine life. Write an essay in 600 words analyzing the effects of global warming on marine life.

Building blocks of a well-constructed prompt



Input data: Any piece of information provided as part of prompt

Example:

You have been provided with a data set containing temperature records and measurements of sea levels in the Pacific Ocean. Write an essay in 600 words analyzing the effect of global warming on marine life in the Pacific Ocean.

Building blocks of a well-constructed prompt

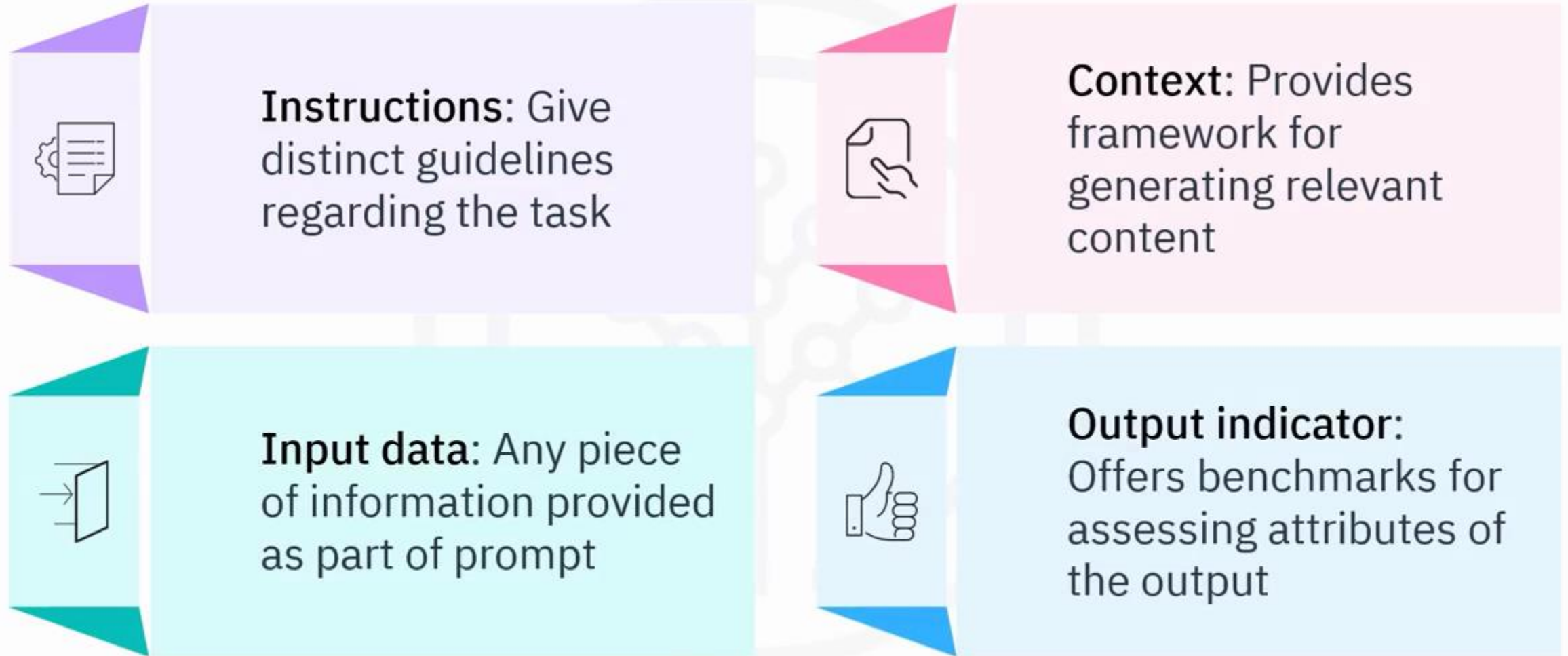


Output indicator: Offers benchmarks for assessing attributes of the output

Example:

The output generated should be an essay of 600 words. It will be evaluated based on the clarity of analysis and incorporation of relevant data or case studies.

Building blocks of a well-constructed prompt



What Is Prompt Engineering?

Prompt engineering

Process of designing effective prompts.

Critical analysis

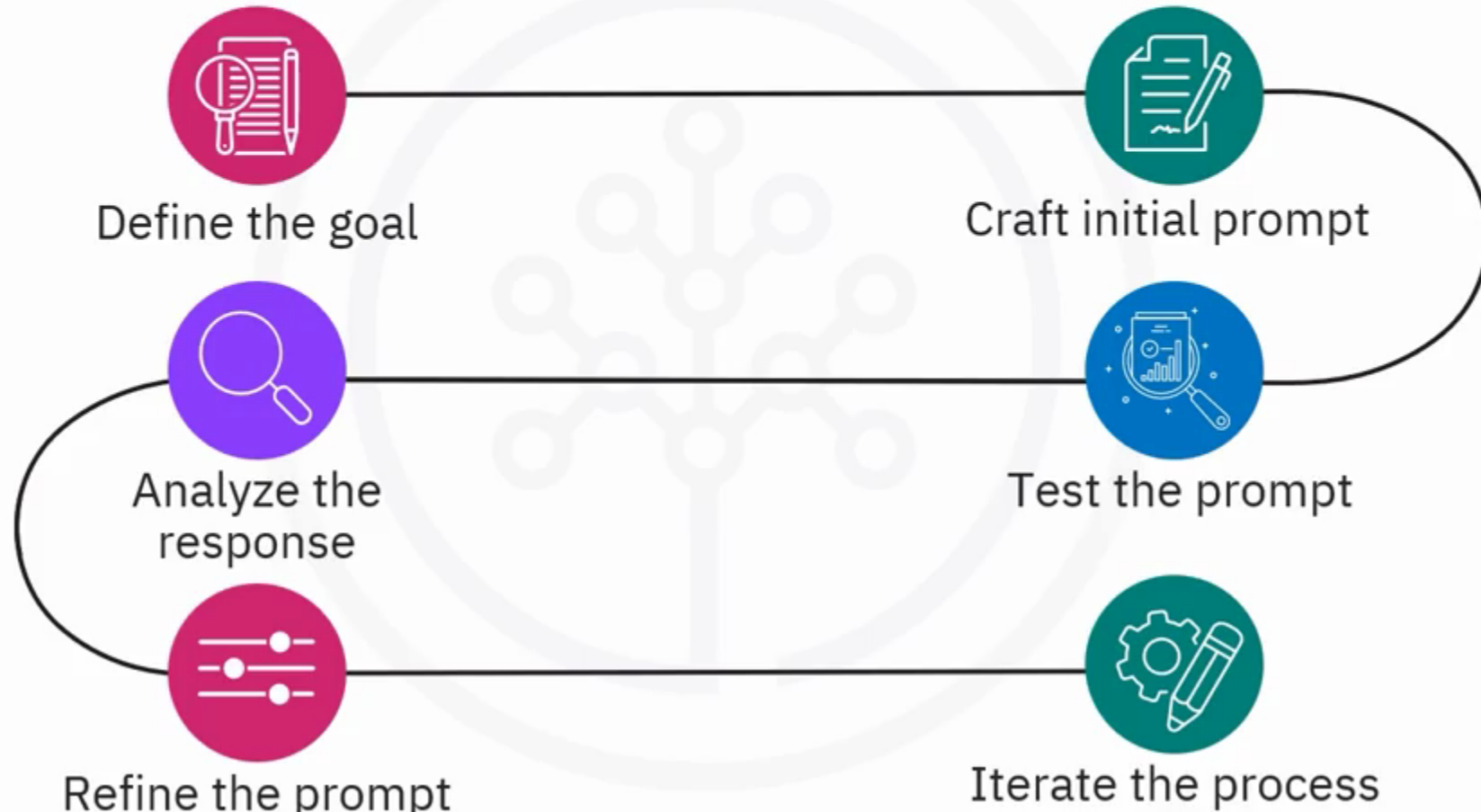
Creativity

Technical acumen



Process involved in prompt engineering

Well-structured iterative process



Process involved in prompt engineering



Define the goal:

Example: Form a brief overview of the benefits and risks associated with artificial intelligence in automobiles.



Craft initial prompt

Example: Write an article that presents a well-rounded analysis of the benefits and drawbacks associated with the incorporation of artificial intelligence in the field of automobile industry.

Process involved in prompt engineering



Test the prompt

Example: Write an article that presents a well-rounded analysis of the benefits and drawbacks associated with the incorporation of artificial intelligence in the field of automobile industry.



Benefits



Drawbacks



Ethical concerns



Positive and negative implications

Process involved in prompt engineering



Analyze the response

Fails to cover: Comprehensive range of benefits and risks associated with artificial intelligence in the automobile industry.

Process involved in prompt engineering



Refine the prompt

Example: Write an informative article discussing the role of artificial intelligence in revolutionizing the automobile industry.

- Address key aspects such as benefits, drawbacks, ethical considerations, and both positive and negative implications.
- Cover specific domains like autonomous driving and real-time traffic analysis, while also examining potential challenges such as technical complexity and cybersecurity concerns.

Process involved in prompt engineering



Iterate the process:

Example: Write an article highlighting how artificial intelligence is reshaping the automobile industry focusing on the positive advancements, particularly in autonomous driving and real-time traffic analysis, while thoroughly exploring concerns related to intricate technical aspects such as decision-making algorithms and potential cybersecurity breaches. Emphasize the implications these concerns may have on vehicle safety. Ensure that the analysis is thorough, backed with examples, and encourages critical thinking.



In-context learning (ICL) – zero shot inference

Prompt



Model



Completion

Classify this review:
I loved this restaurant!
Sentiment: Positive

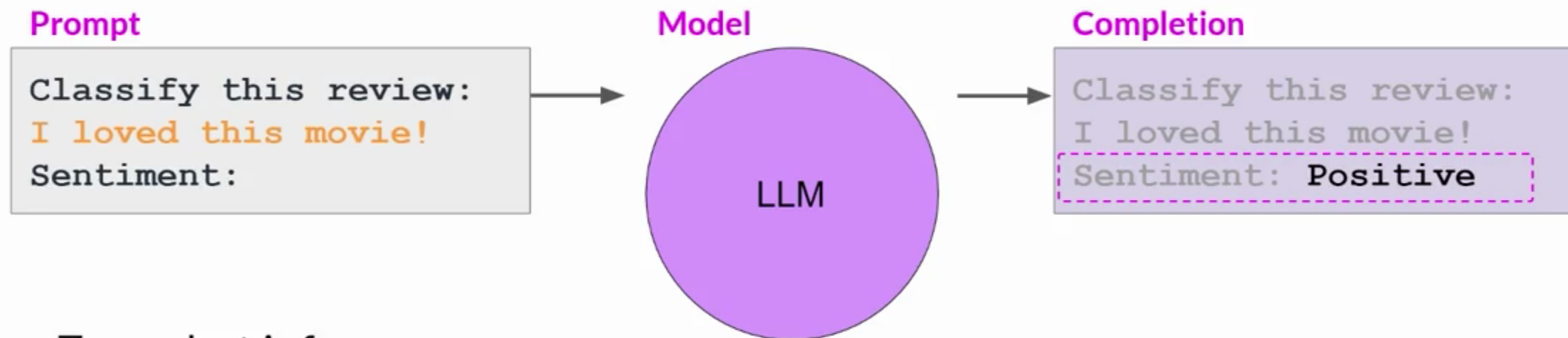
Classify this review: I don't like the environment
the restaurant has. The tables look old and oily.
Sentiment:

LLM

The sentiment of this review is **negative**.
The reviewer expresses dissatisfaction with
the restaurant's environment, specifically
mentioning the old and oily tables.

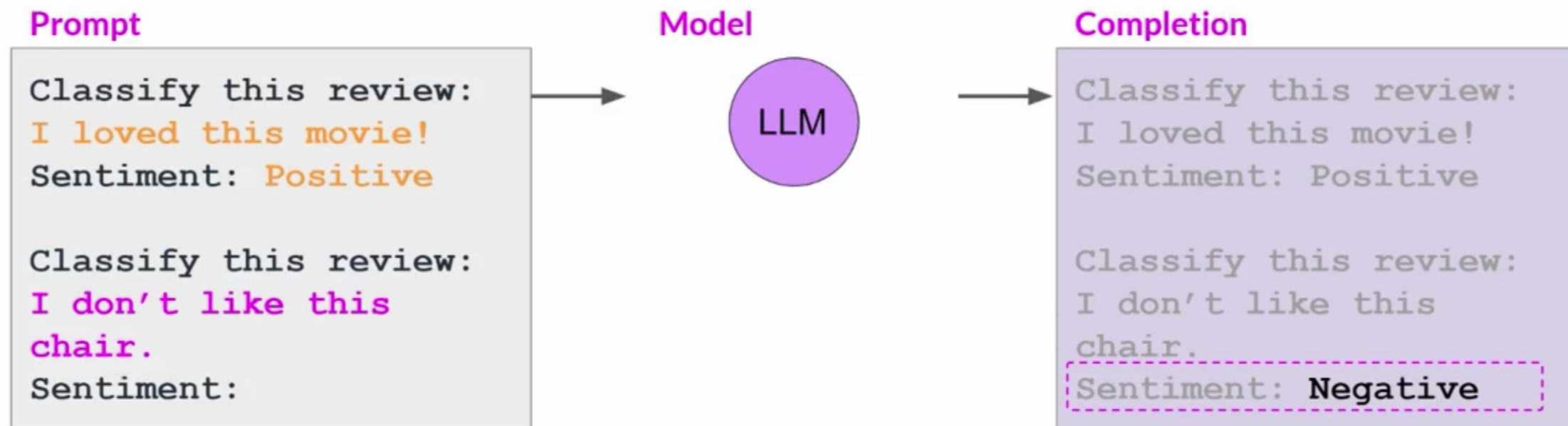
Zero-shot Inference

In-context learning (ICL) - zero shot inference



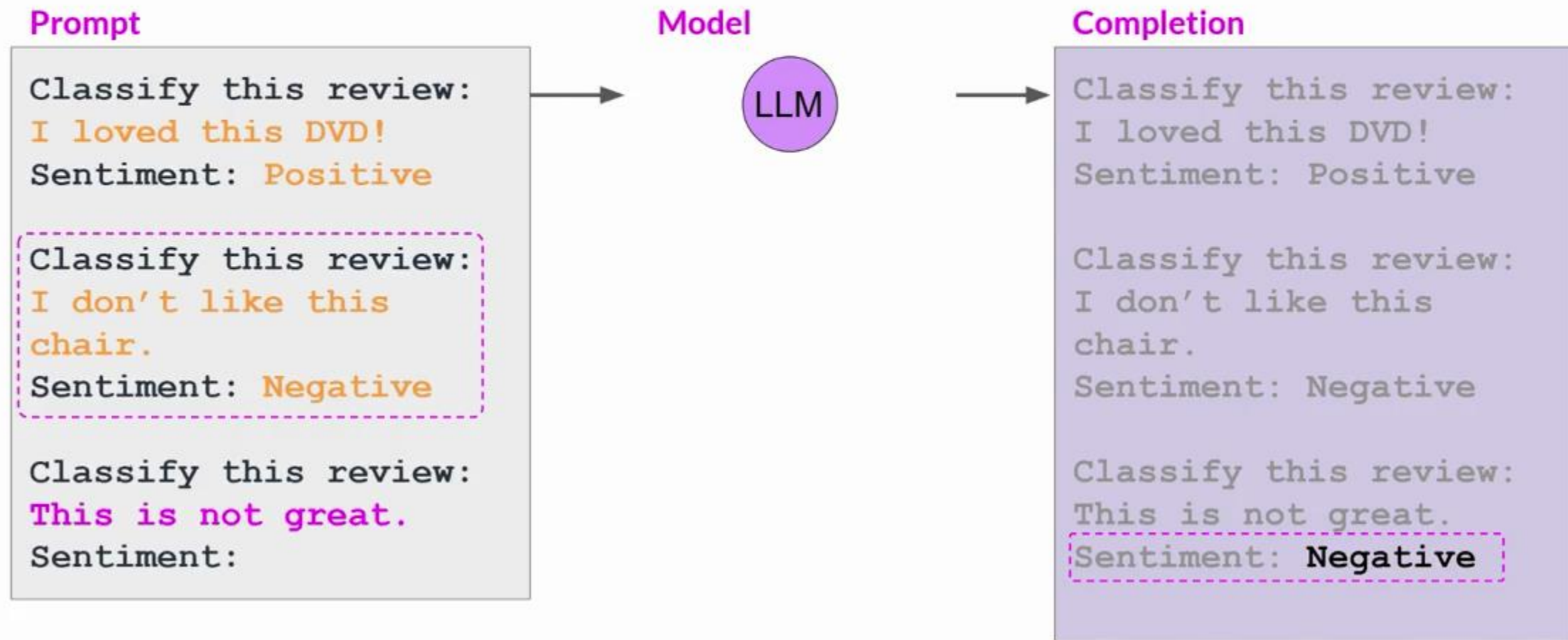
Zero-shot inference

In-context learning (ICL) - one shot inference



One-shot inference

In-context learning (ICL) - few shot inference



Summary of in-context learning (ICL)

Prompt // Zero Shot

Classify this review:
I loved this movie!
Sentiment:

Prompt // One Shot

Classify this review:
I loved this movie!
Sentiment: Positive

Classify this review:
I don't like this
chair.
Sentiment:

Prompt // Few Shot >5 or 6 examples

Classify this review:
I loved this movie!
Sentiment: Positive

Classify this review:
I don't like this
chair.
Sentiment: Negative

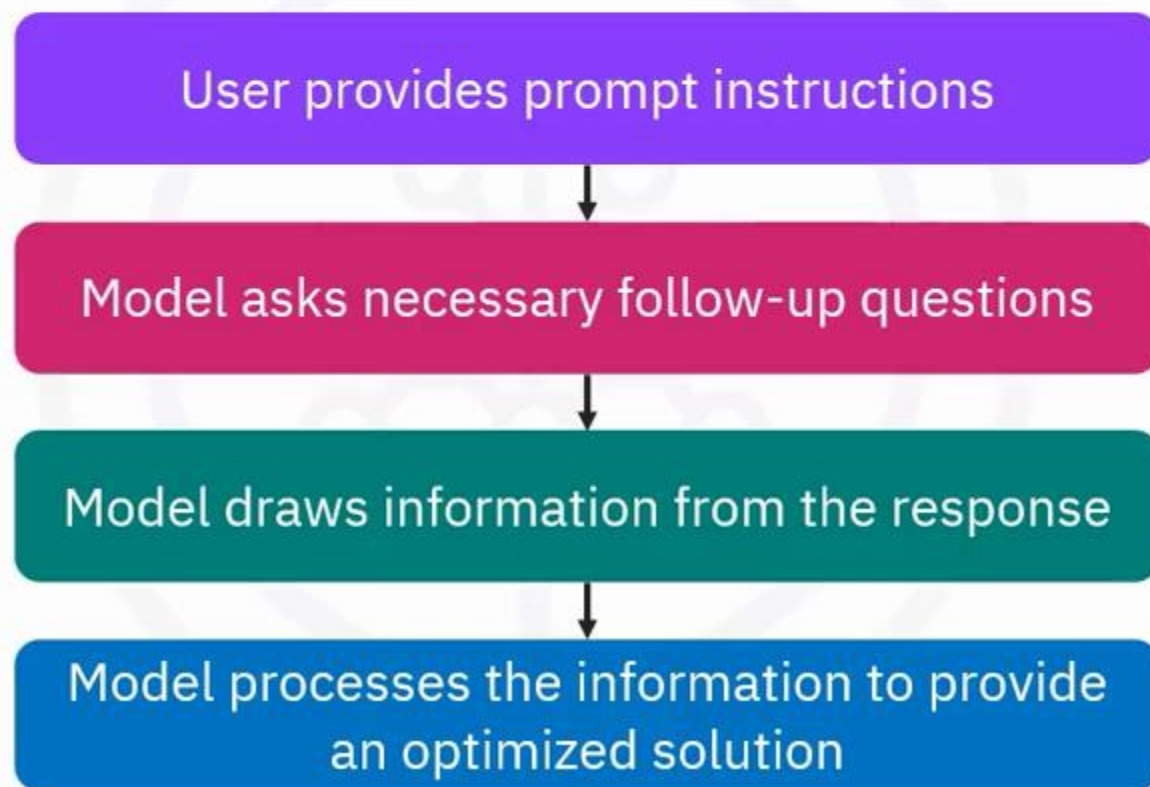
Classify this review:
Who would use this
product?
Sentiment:

Context Window
(few thousand words)

Interview Pattern Approach

Working behind the approach

The interview pattern approach involves designing prompts by simulating a conversation or interacting in the interview style.



Example

Prompt instructions:

“You will act as a seasoned travel expert. Your objective is to engage in a comprehensive trip-planning session with me. Begin by asking a series of detailed questions, one at a time, to gather all the essential information required to craft the most tailored and memorable travel itinerary based on my specific preferences, interests, and budget.”

Example

Follow-up questions:

01

What types of destinations do you enjoy traveling to the most?

02

Could you describe your ideal vacation in terms of activities and experiences?

03

How do you typically plan your trips, and what factors are most important to you when choosing a destination?

04

Do you find any specific cultural or historical aspects intriguing when planning your travel destination?

05

What kind of accommodation options do you prefer when you travel, and why?

06

How do you balance budget considerations with the desire for a memorable travel experience?

Chain-of-Thought Approach

Working behind the approach

Chain-of-Thought is a prompt-based learning approach.



Working behind the approach

It involves breaking down a complex task into smaller and easier ones through a sequence of more straightforward prompts.



Working behind the approach

Feed the model with related questions along with their corresponding solutions.

Prompt includes:

- Related question
- Accurate solution to the question
- Another question based on the same reasoning

Example

Mathew has 6 eggs. He buys 2 more trays of eggs. Each tray has 12 eggs. How many eggs does he have now?

Question: Mary has 8 radishes. She used 5 radishes to prepare the dinner. The next morning, she bought 10 more radishes. How many radishes does she have now?

Solution: Mary had 8 radishes. She cooked dinner using 5 of them. So, she had $8 - 5 = 3$ radishes left with her. The next morning, she bought 10 more. So, she has $3 + 10 = 13$ radishes now.

Example

Model Input

Q. Mary has 8 radishes. She used 5 radishes to prepare the dinner. The next morning, she bought 10 more radishes. How many radishes does she have now?

A. Mary had 8 radishes. She cooked dinner using 5 of them. So, she had $8 - 5 = 3$ radishes left with her. The next morning, she bought 10 more. So, she has $3 + 10 = 13$ radishes now.

Q. Mathew has 6 eggs. He buys 2 more trays of eggs. Each tray has 12 eggs. How many eggs does he have now?

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The answer is 8. **X**

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. **✓**

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

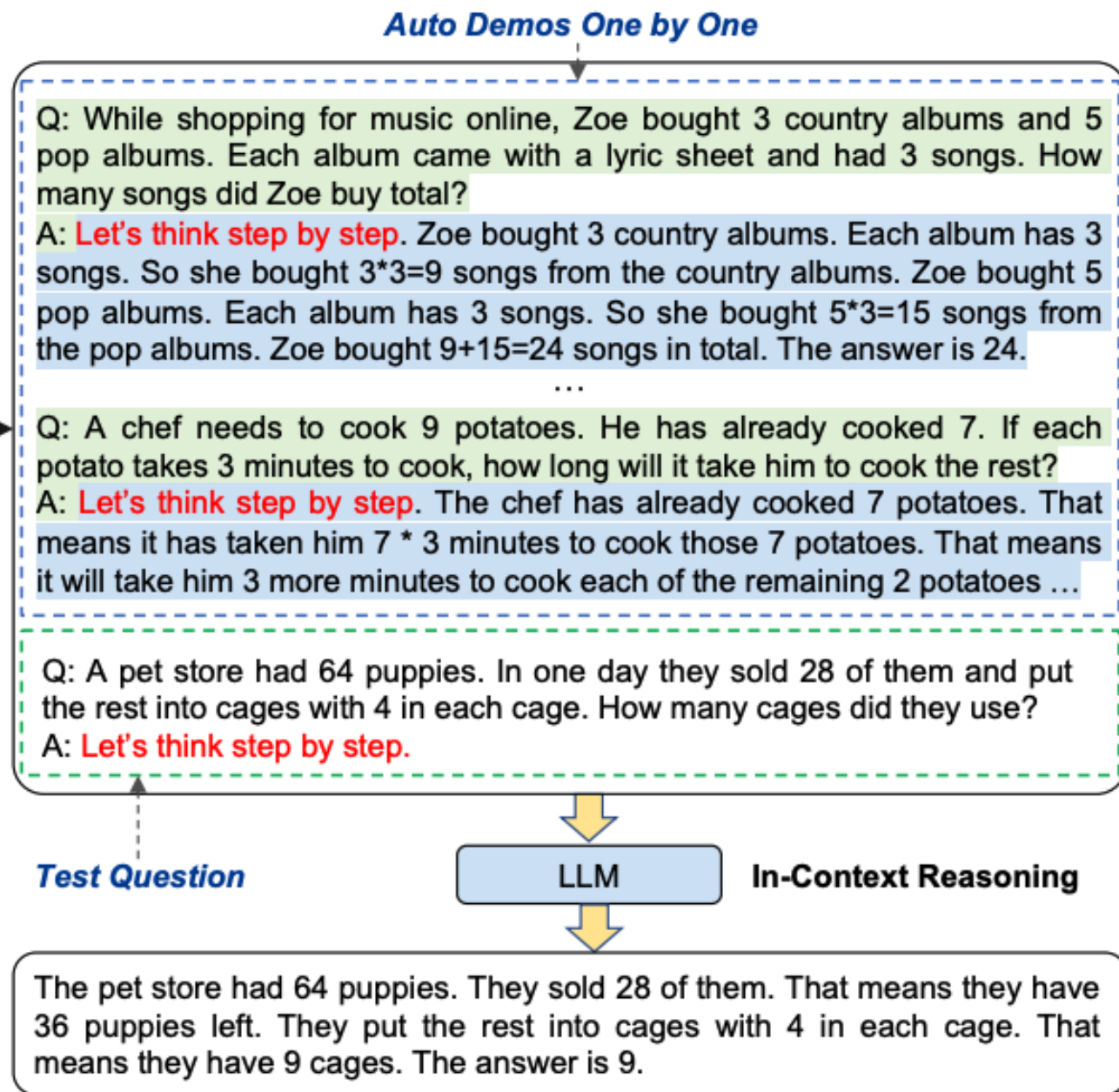
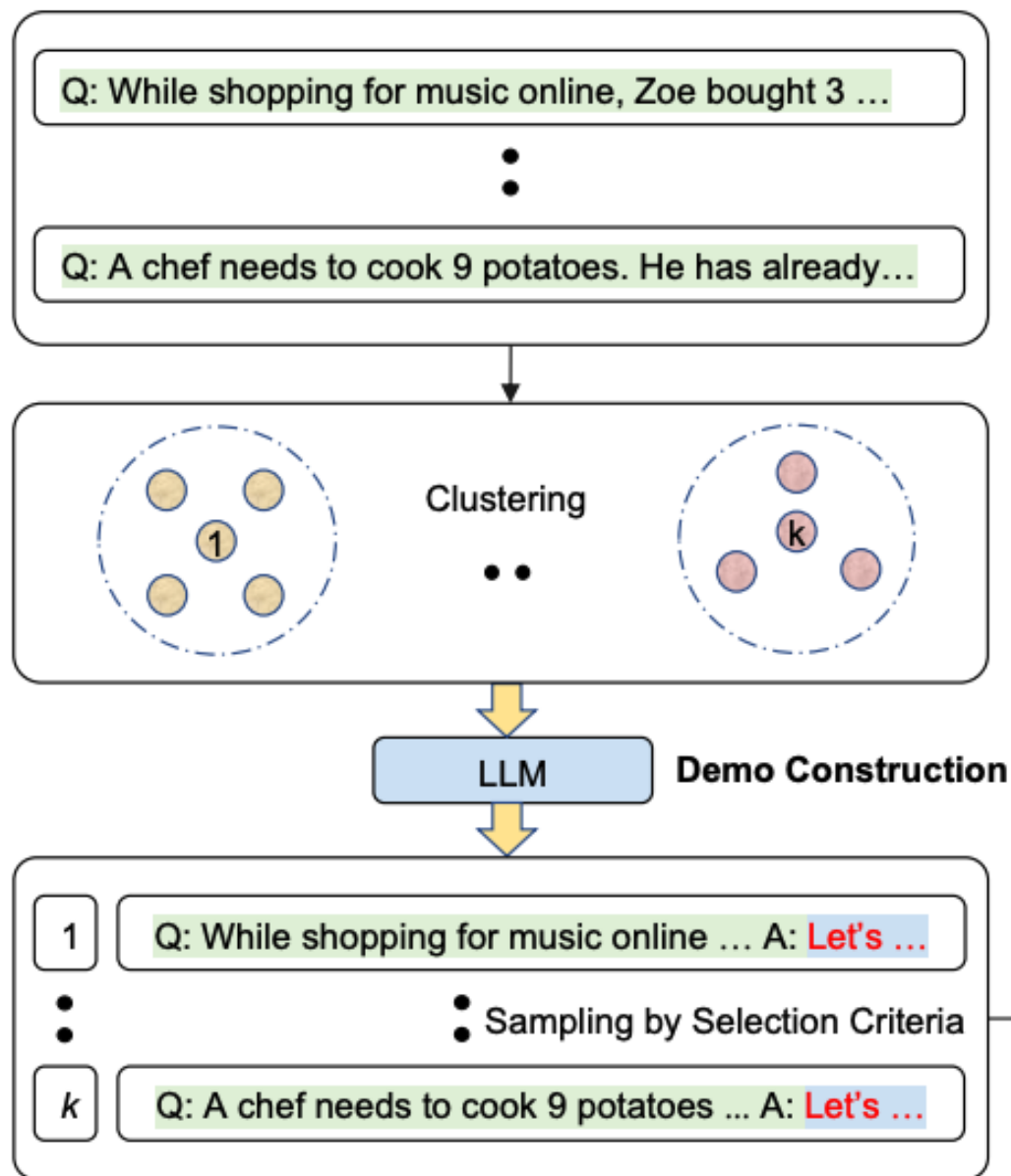
(Output) 8 **X**

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. **✓**



Tree-of-Thought Approach

Working behind the approach

Tree-of-Thought is built to expand the capabilities of Chain-of-Thought.



Enables generative AI models to demonstrate advanced reasoning capabilities



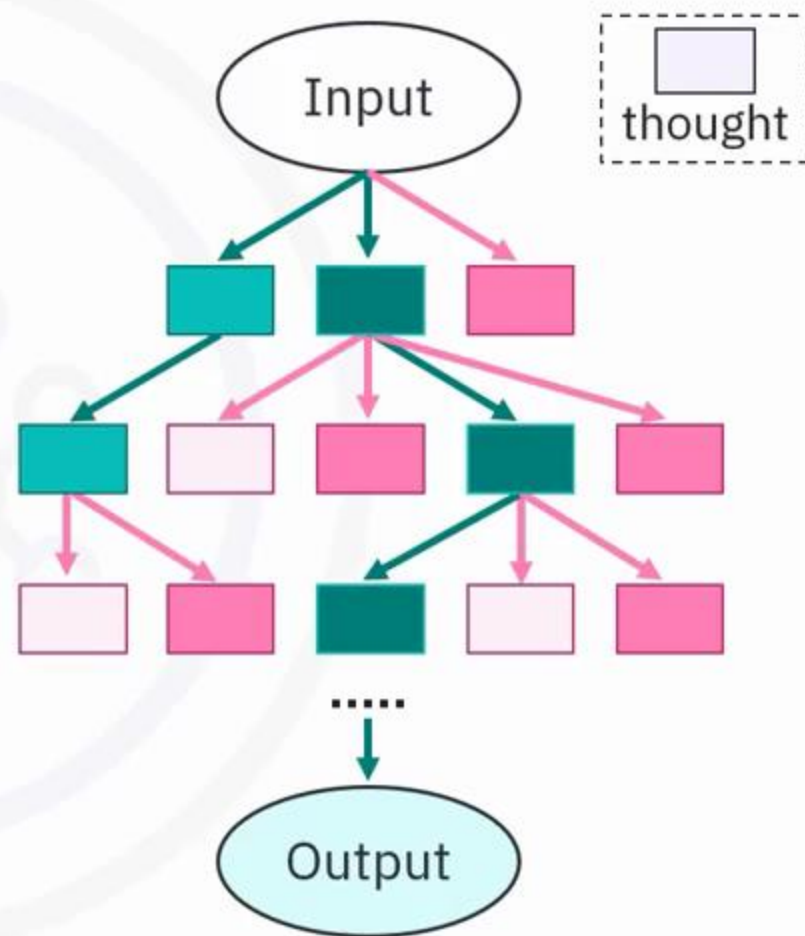
Involves hierarchically structuring a prompt or query, akin to a tree structure



Holds immense potential for unlocking new solutions and tackling complex problems

Working behind the approach

- Involves generating multiple lines of thought, resembling a decision tree.
- Allows the model to evaluate and pursue multiple paths simultaneously.
- Each thought or idea branches out, creating a tree-like structure of interconnected thoughts.



Example

Design recruitment and retention strategies for attracting skilled remote employees



Example

Prompt instructions:

Imagine three different experts answering this question. All experts will write down 1 step of their thinking, and then share it with the group. Then all experts will go on to the next step, etc. If any expert realizes they're wrong at any point, then they leave.

Prompt:

Act as a human resource specialist, design a recruitment and retention strategy for an e-commerce business, focusing on attracting and retaining skilled remote employees.



REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao^{*,1}, Jeffrey Zhao², Dian Yu², Nan Du², Izhak Shafran², Karthik Narasimhan¹, Yuan Cao²

¹Department of Computer Science, Princeton University

²Google Research, Brain team

¹{shunyuy, karthikn}@princeton.edu

²{jeffreyzhao, dianyu, dunan, izhak, yuancao}@google.com

ABSTRACT

While large language models (LLMs) have demonstrated impressive performance across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied as separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions in an interleaved manner, allowing for greater synergy between the two: reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with and gather additional information from external sources such as knowledge bases or environments. We apply our approach, named ReAct, to a diverse set of language and decision making tasks and demonstrate its effectiveness over state-of-the-art baselines in addition to improved human interpretability and trustworthiness. Concretely, on question answering (HotpotQA) and fact verification (Fever), ReAct overcomes prevalent issues of hallucination and error propagation in chain-of-thought reasoning by interacting with a simple Wikipedia API, and generating human-like task-solving trajectories that are more interpretable than baselines without reasoning traces. Furthermore, on two interactive decision making benchmarks (ALFWorld and WebShop), ReAct outperforms imitation and reinforcement learning methods by an absolute success rate of 34% and 10% respectively, while being prompted with only one or two in-context examples.



(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: Search[Front Row (software)]

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: Finish[keyboard function keys]





ReAct Example

Question What is the elevation range for the area that the eastern sector of the Colorado orogeny extends into?

- Thought 1 I need to search Colorado orogeny, find the area that the eastern sector of the Colorado orogeny extends into, then find the elevation range of the area.

- Action 1 Search[Colorado orogeny]

- Observation 1 The Colorado orogeny was an episode of mountain building (an orogeny) in Colorado and surrounding areas.

- Thought 2 It does not mention the eastern sector. So I need to look up eastern sector.

- Action 2 Lookup[eastern sector]

- Observation 2 (Result 1 / 1) The eastern sector extends into the High Plains and is called the Central Plains orogeny.

- Thought 3 The eastern sector of Colorado orogeny extends into the High Plains. So I need to search High Plains and find its elevation range.

Action 3 Search[High Plains]

Observation 3 High Plains refers to one of two distinct land regions

Thought 4 I need to instead search High Plains (United States).

Action 4 Search[High Plains (United States)]

Observation 4 The High Plains are a subregion of the Great Plains. From east to west, the High Plains rise in elevation from around 1,800 to 7,000 ft (550 to 2,130 m).[3]

Thought 5 High Plains rise in elevation from around 1,800 to 7,000 ft, so the answer is 1,800 to 7,000 ft.

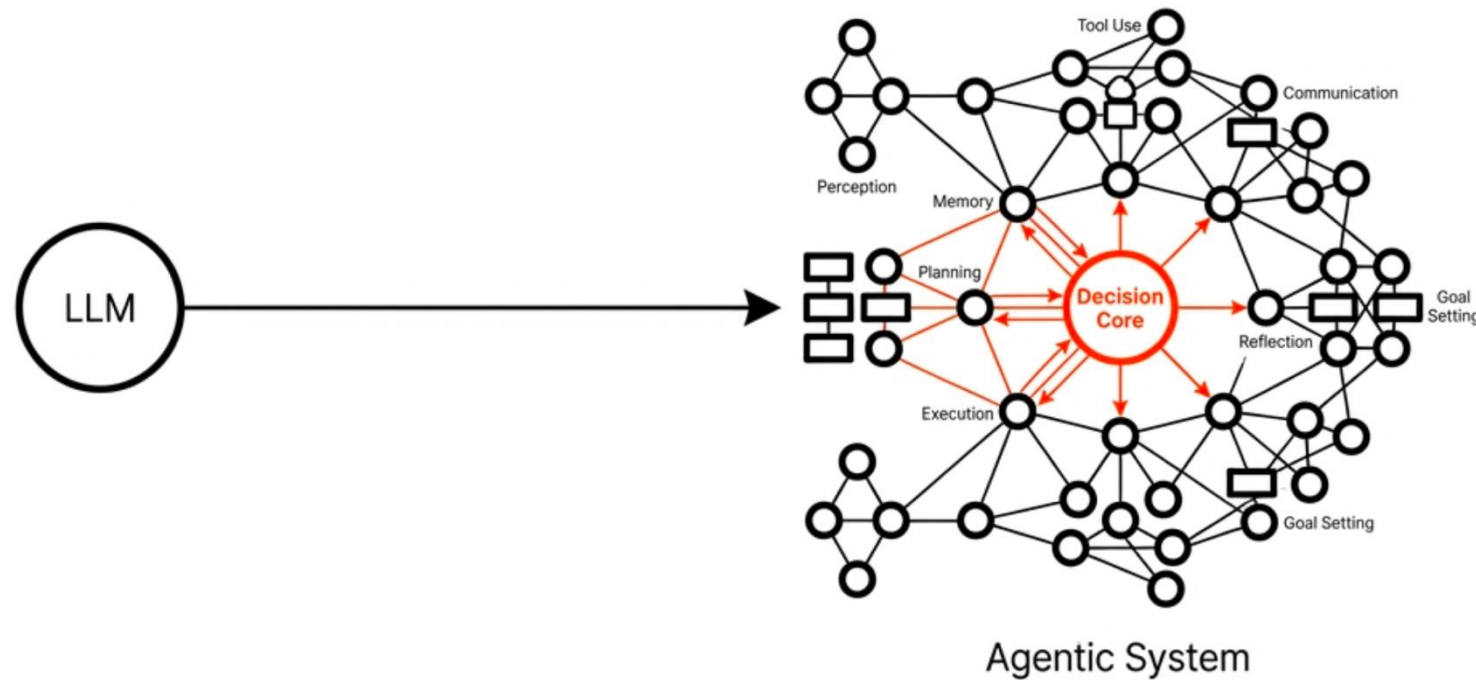
Action 5 Finish[1,800 to 7,000 ft]

...



A Unified Taxonomy for Advanced Agentic Architectures

From Isolated Reasoning to Agentic Collaboration

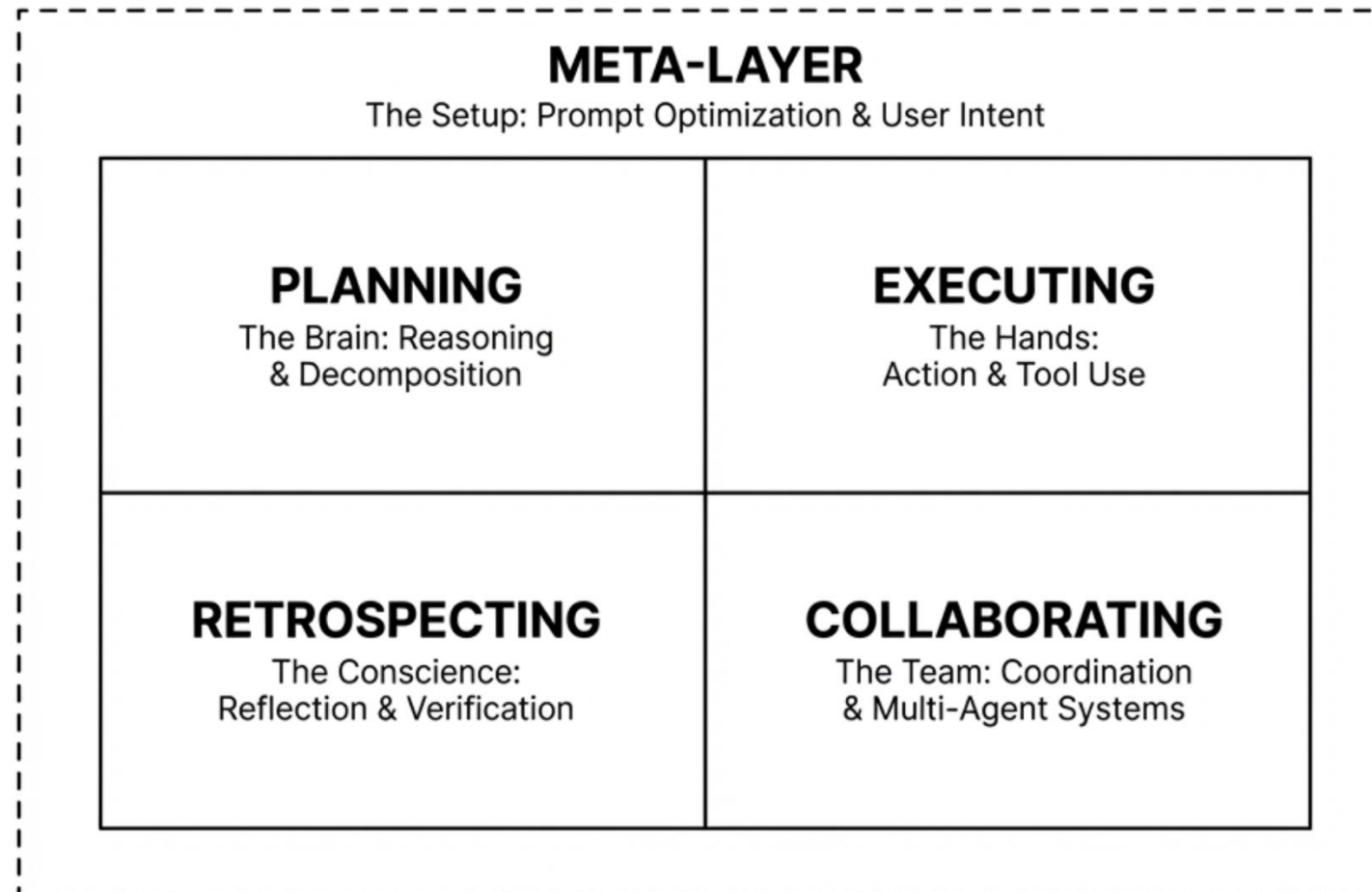


State of the Art Techniques in Agent Engineering



The Four Pillars of Agentic Architecture

Advanced prompting techniques function as subcategories, intersections, or wrappers around these four core cognitive processes.





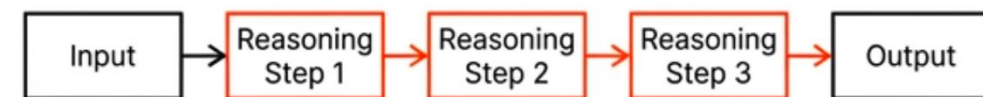
Planning: Structuring the Cognitive Process

Moving from intuitive guessing to analytical reasoning.

Standard Prompting



Chain-of-Thought (CoT)

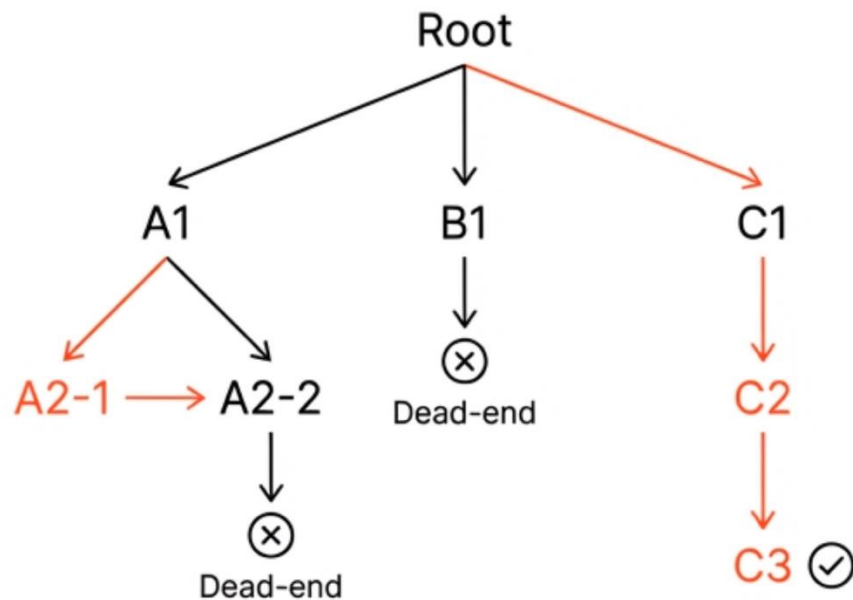


- Chain-of-Thought (CoT): Generates intermediate reasoning steps (Wei et al., 2022).
- Zero-Shot CoT: Triggers reasoning via the append command “Let’s think step by step” (Kojima et al., 2022).



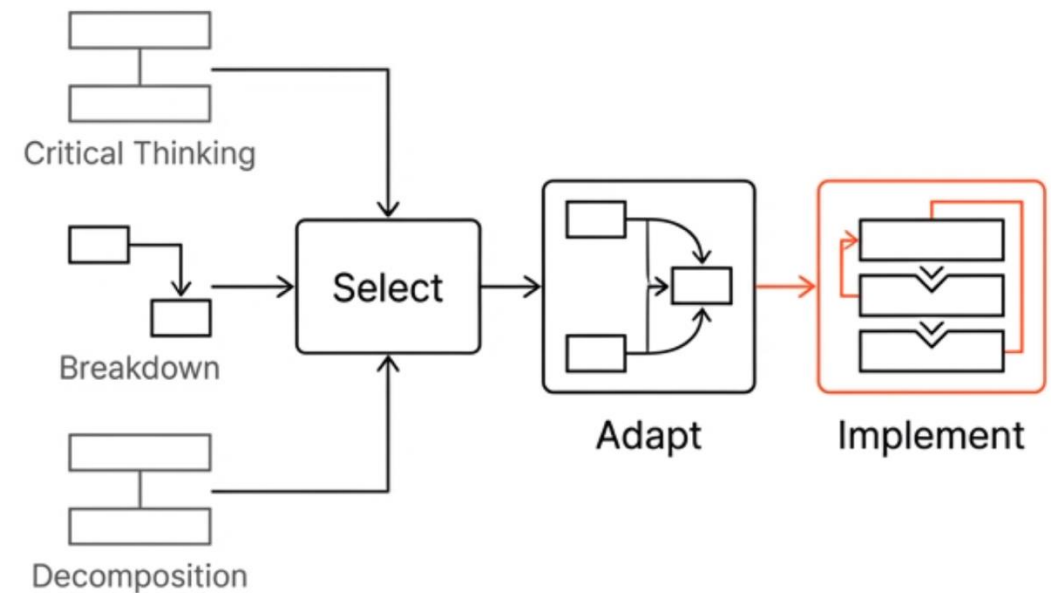
Advanced Decomposition & Non-Linear Reasoning

Tree of Thoughts (ToT)



Exploration, Look-ahead, & Backtracking

Self-Discover

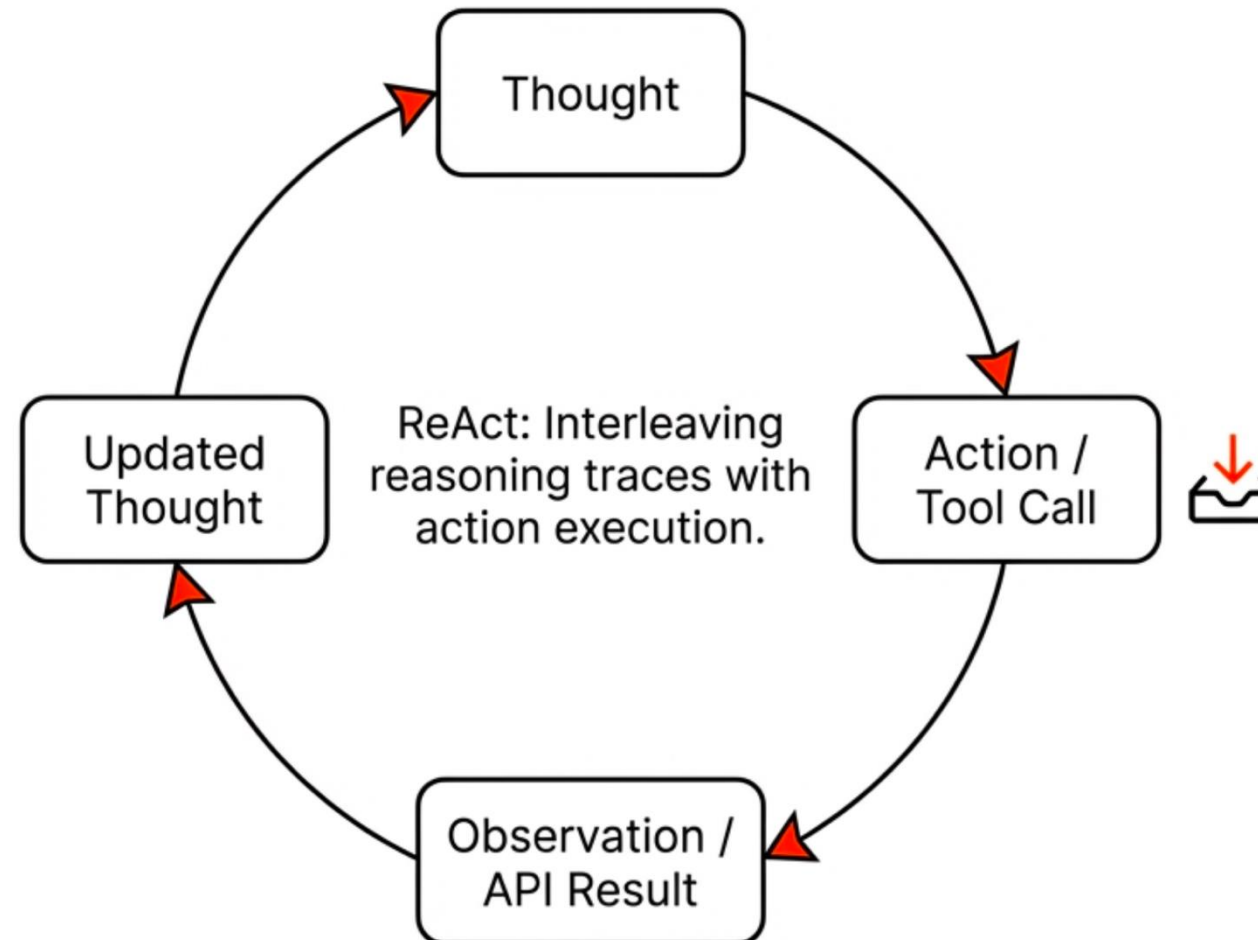


LLM Self-Composes Reasoning Structure



Executing: The Thought-Action Loop

Grounding LLMs with tools and external observation.

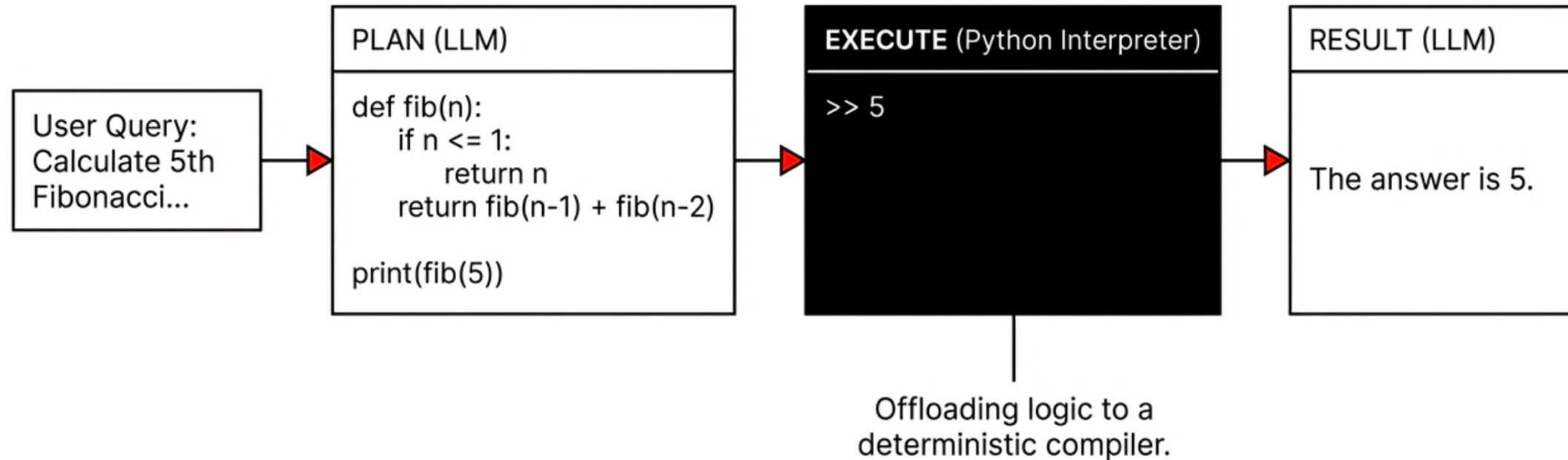




Neuro-Symbolic Execution

Disentangling computation from semantic reasoning.

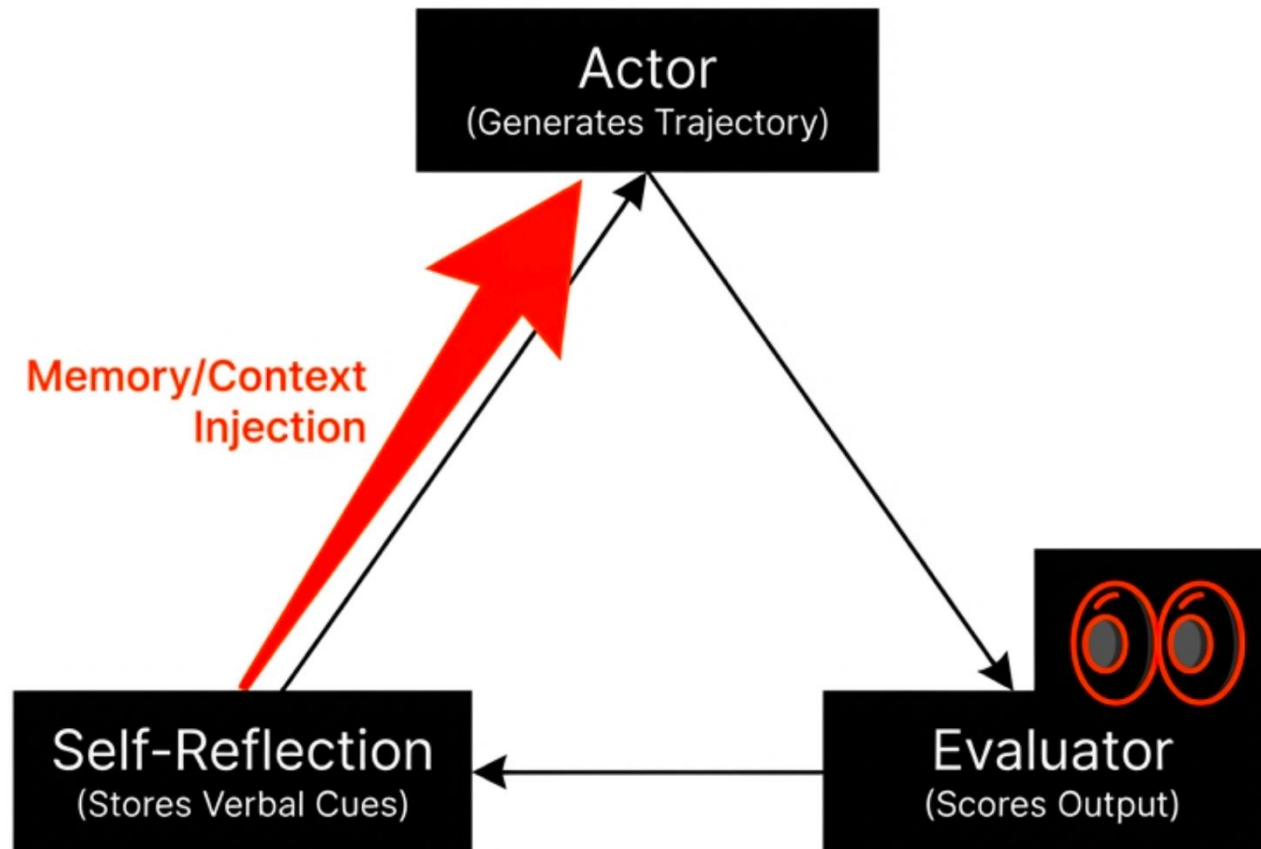
Chain-of-Code





Retrospecting: The Feedback Loop

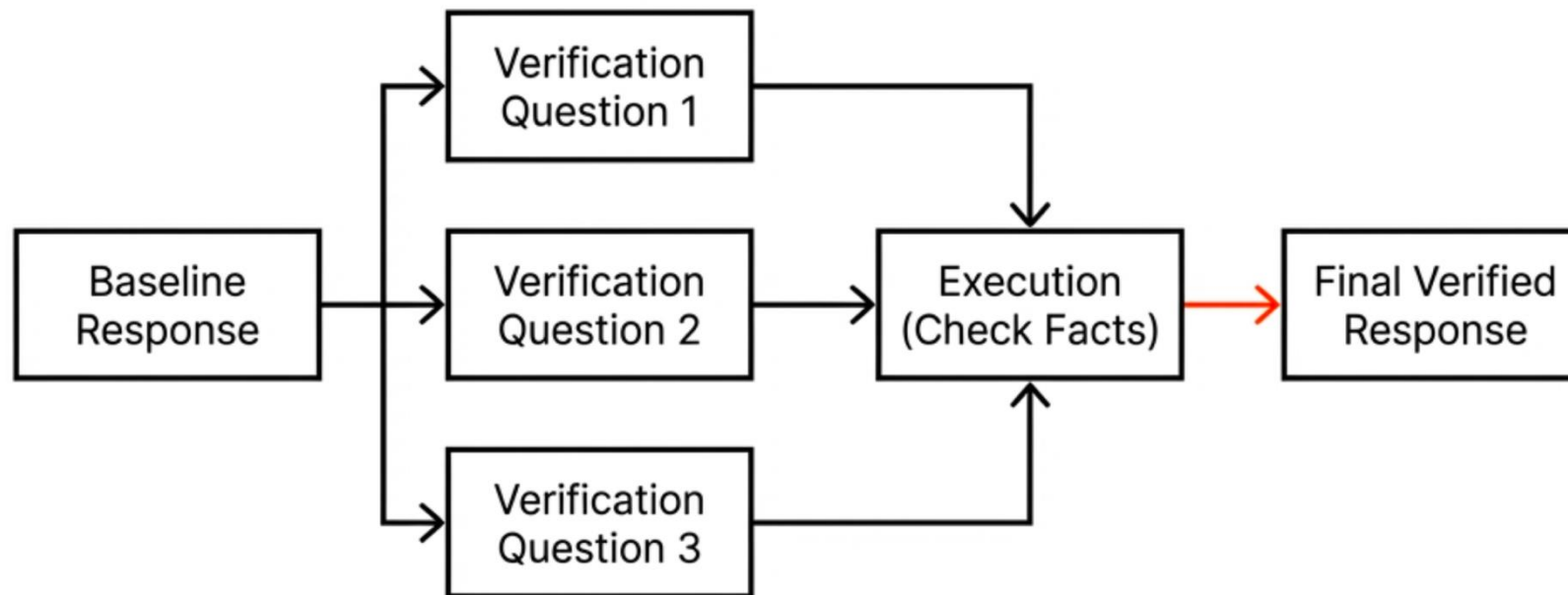
Converting transient experience into improved performance.





Verification & Hallucination Reduction

Chain-of-Verification (CoVe)



Chain-of-Note

For RAG Systems:
The model explicitly
notes relevance
before answering.



Data
Found

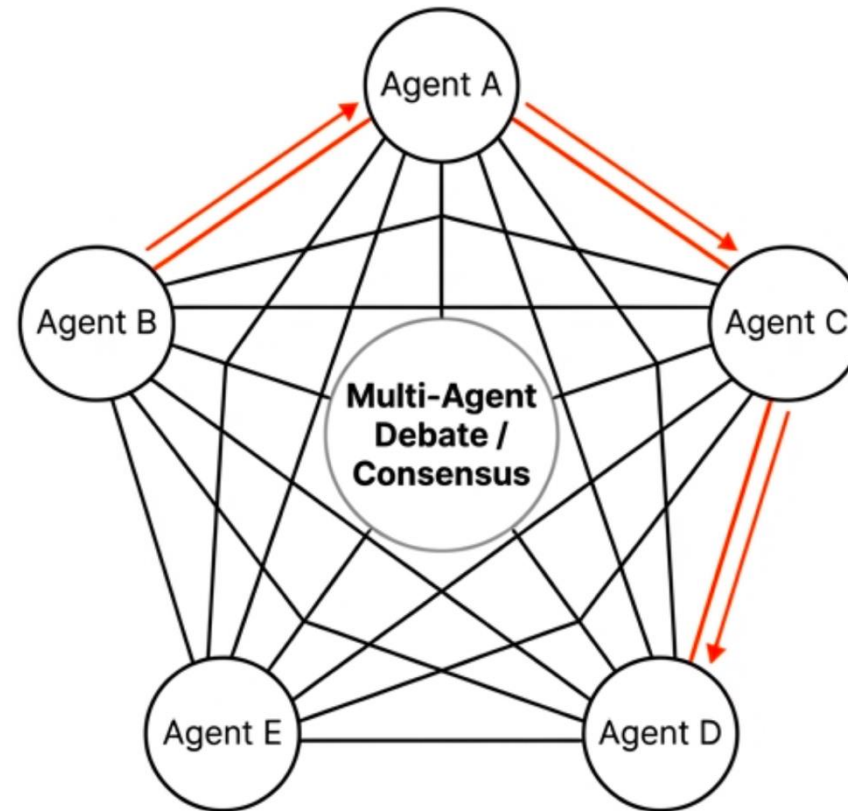


Data
Missing



Collaborating: Scaling Intelligence

Overcoming individual bias through multi-agent systems.

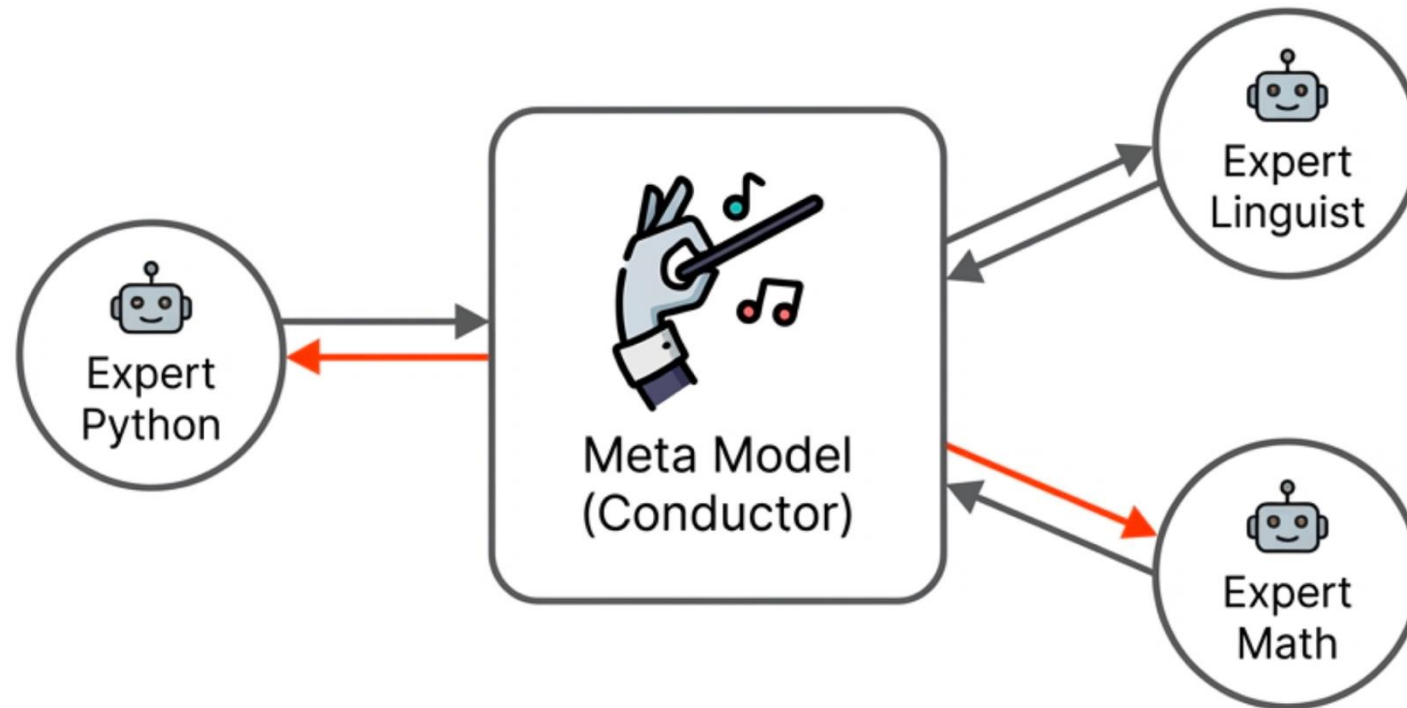


- Divergent thinking leads to convergent accuracy.
- AgentsNet / DyLAN: Dynamic topologies for distributed problem solving.



Meta-Prompting: The Orchestrator

Turning a single model into a conductor and a panel of experts.

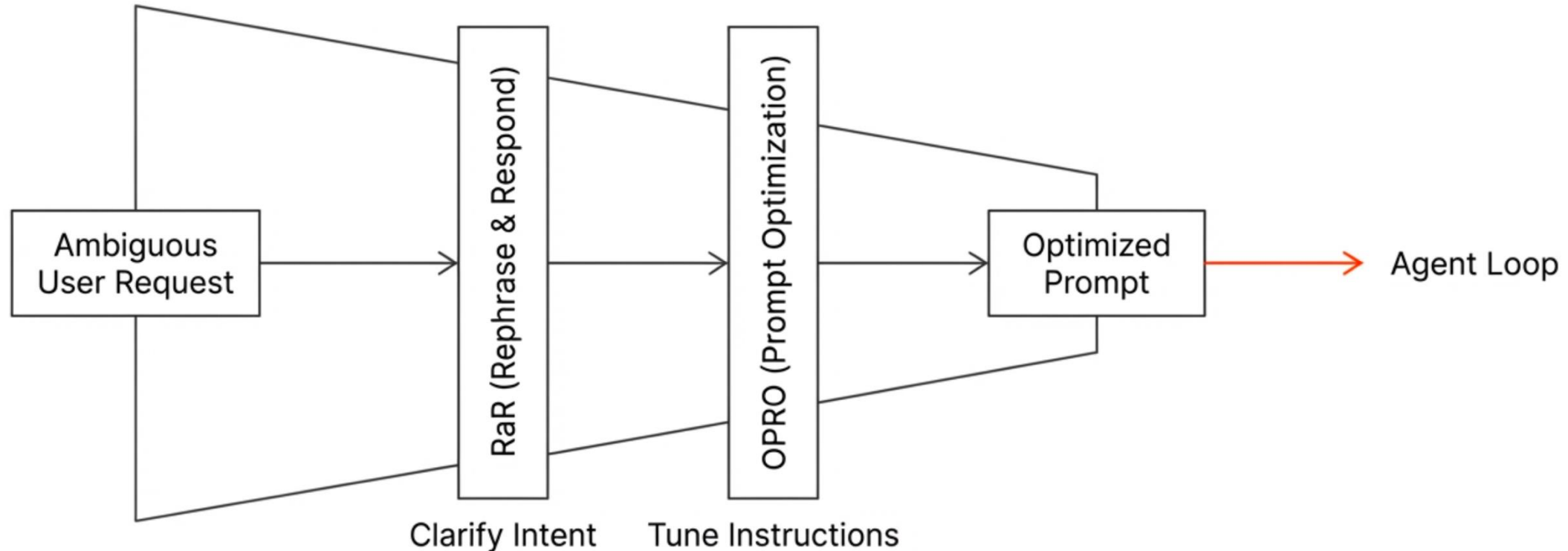


Note: The Meta Model maintains history. Experts have no memory.



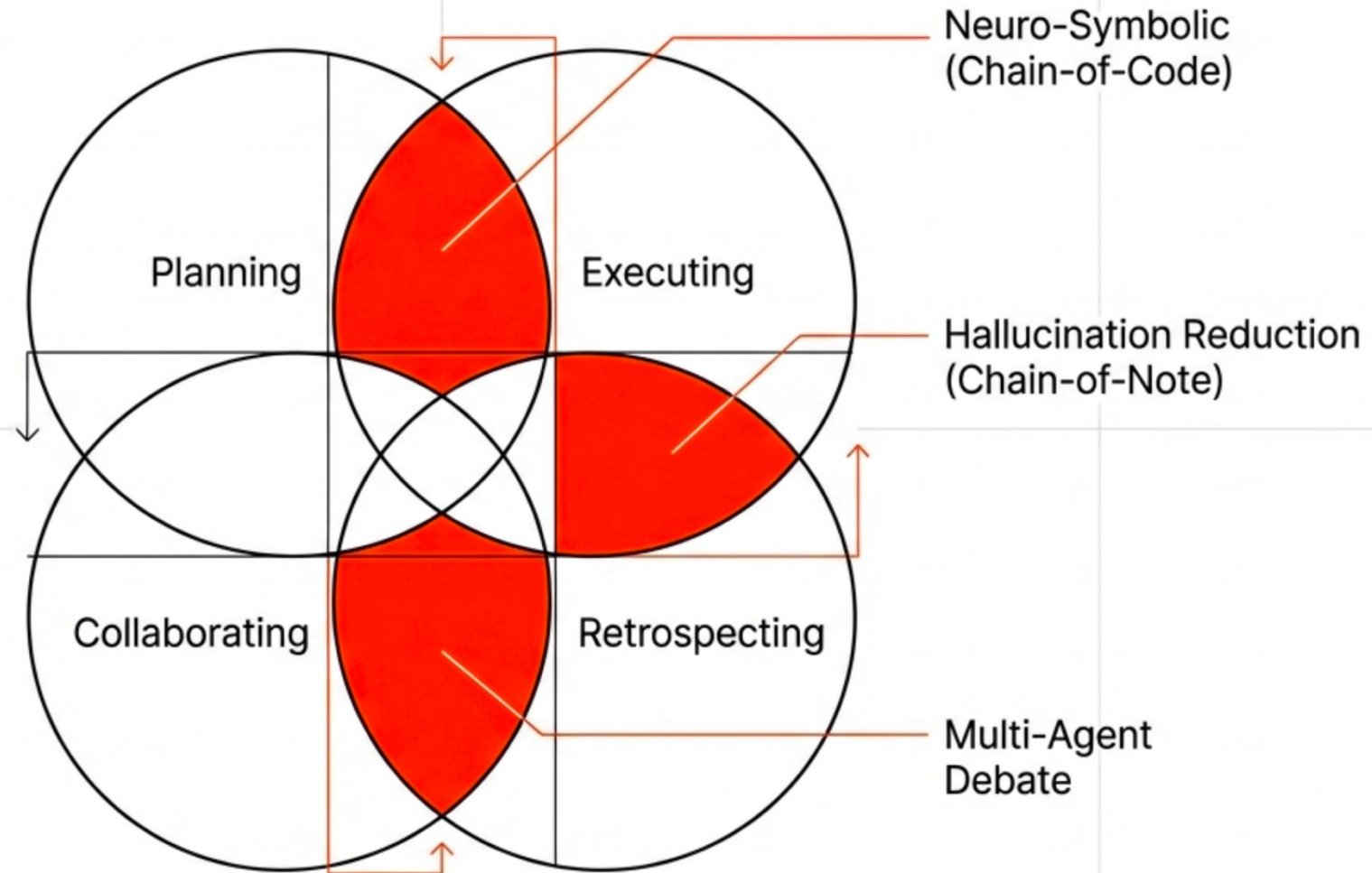
The Meta-Layer: Optimization & Intent

Pre-processing steps to ensure the agent solves the right problem.





Synthesis: Advanced Hybrids



The most robust architectures are not single techniques, but composite workflows.



From Prompt Engineering to Agent Engineering



“We are moving away from crafting text for a model, and toward architecting the cognitive environment in which the model operates.”



Overview

LangChain

- Open-source development framework for LLM applications
- Python and JavaScript (TypeScript) packages
- Focused on composition and modularity

Key value adds:

- Modular components
- Use cases: Common ways to combine components



Why use prompt templates?

```
prompt = ""
```

```
Your task is to determine if  
the student's solution is  
correct or not.
```

```
To solve the problem do the following:  
- First, work out your own solution to the problem.  
- Then compare your solution to the student's solution  
and evaluate if the student's solution is correct or not.
```

```
...  
Use the following format:
```

```
Question:
```

```
...
```

```
question here
```

```
...
```

```
Student's solution:
```

```
...
```

```
student's solution here
```

```
...
```

```
Actual solution:
```

```
...
```

```
...
```

```
steps to work out the solution and your solution here
```

```
...
```

```
Is the student's solution the same as actual solution \  
just calculated:
```

```
...
```

```
yes or no
```

```
...
```

```
Student grade:
```

```
...
```

```
correct or incorrect
```

```
...
```

```
Question:
```

```
...
```

```
{question}
```

```
...
```

```
Student's solution:
```

```
...
```

```
{student_solution}
```

```
...
```

```
Actual solution:
```

```
...
```

Prompts can be
long and detailed.

Reuse good
prompts when
you can!

LangChain also
provides prompts
for common
operations.



LangChain output parsing works with prompt templates

```
EXAMPLES = ["""\nQuestion: What is the elevation range\nfor the area that the eastern sector\nof the Colorado orogeny extends into?\n\nThought: need to search Colorado orogeny, find\nthe area that the eastern sector of the Colorado\norogeny extends into, then find the elevation range\nof the area.\n\nAction: Search[Colorado orogeny]\n\nObservation: The Colorado orogeny was an\nepisode of mountain building (an orogeny) in\nColorado and surrounding areas.\n\nThought: It does not mention the eastern sector.\nSo I need to look up eastern sector.\nAction: Lookup[eastern sector]\n\n...\n\nThought: High Plains rise in elevation from\naround 1,800 to 7,000 ft, so the answer is 1,800 to\n7,000 ft.\n\nAction: Finish[1,800 to 7,000 ft]""",\n]
```

LangChain library
functions parse the
LLM's output
assuming that it will
use certain keywords.

Example here uses
Thought, **Action**,
Observation as
keywords for Chain-
of-Thought
Reasoning. (ReAct)

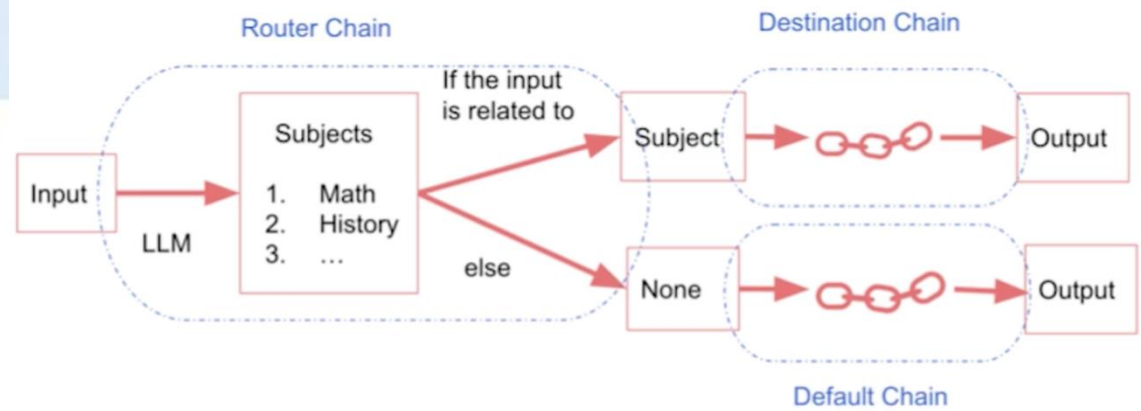


Memory

- Large Language Models are 'stateless'
 - Each transaction is independent
- Chatbots appear to have memory by providing the full conversation as 'context'



- LangChain provides several kinds of 'memory' to store and accumulate the conversation.



LLMChain

- Basic building block for LLM-powered applications
- Combines a prompt template with an LLM

SimpleSequentialChain

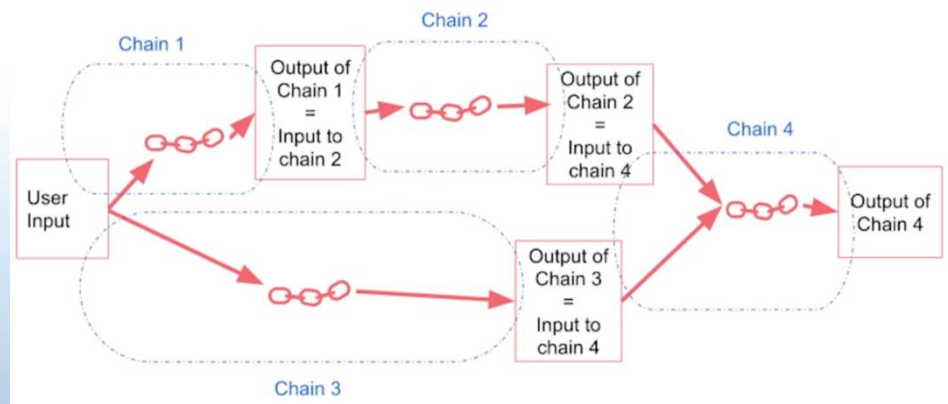
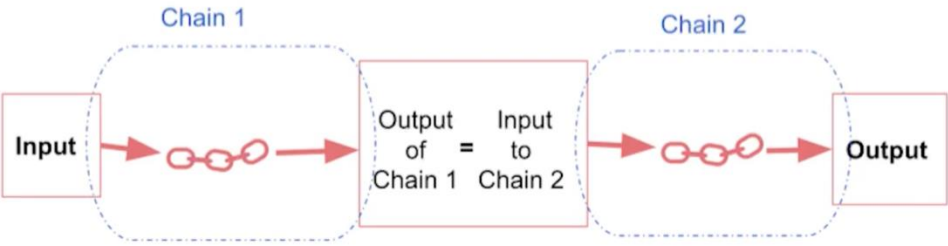
- Chains multiple LLMChains together
- Output of one chain becomes input for the next
- Single input, single output

SequentialChain

- More flexible than SimpleSequentialChain
- Allows multiple inputs and outputs
- Can specify which outputs to use as inputs for subsequent chains

Router Chain

- Dynamically selects which chain to use based on input
- Useful for handling diverse queries or tasks





Transfer Learning in NLP

- Used in standard NLP tasks
 - Question answering
 - Text generation
 - Text summarization
 - Named Entity Recognition
 - Key value pair identification
 - ...
- Transfer learning in NLP through
 - Finetuning language models for down stream tasks
 - Generating contextualized word embeddings



Pre-Training Model Architectures in NLP

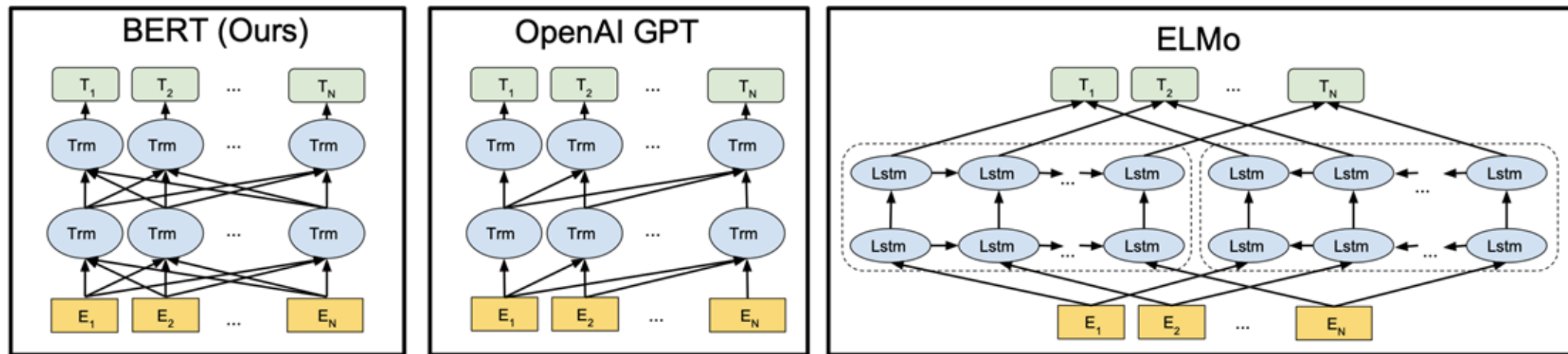


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.



Using Transformer for Transfer Learning in NLP

- Transformers handle long-term dependencies better than LSTMs
- Encoder-Decoder structure of the transformer made it perfect for machine translation
- How to pre-train a language model using transformer-based architecture (with attention mechanisms and faster training due to parallelization) that can be finetuned for downstream supervised learning tasks?



BERT (Bidirectional Encoder Representations from Transformers)

- OpenAI Transformer only trains a forward language model while ELMo's model was bidirectional
- BERT is a transformer-based model whose language model is conditioned on both left and right contexts
- BERT is basically a trained Transformer Encoder stack

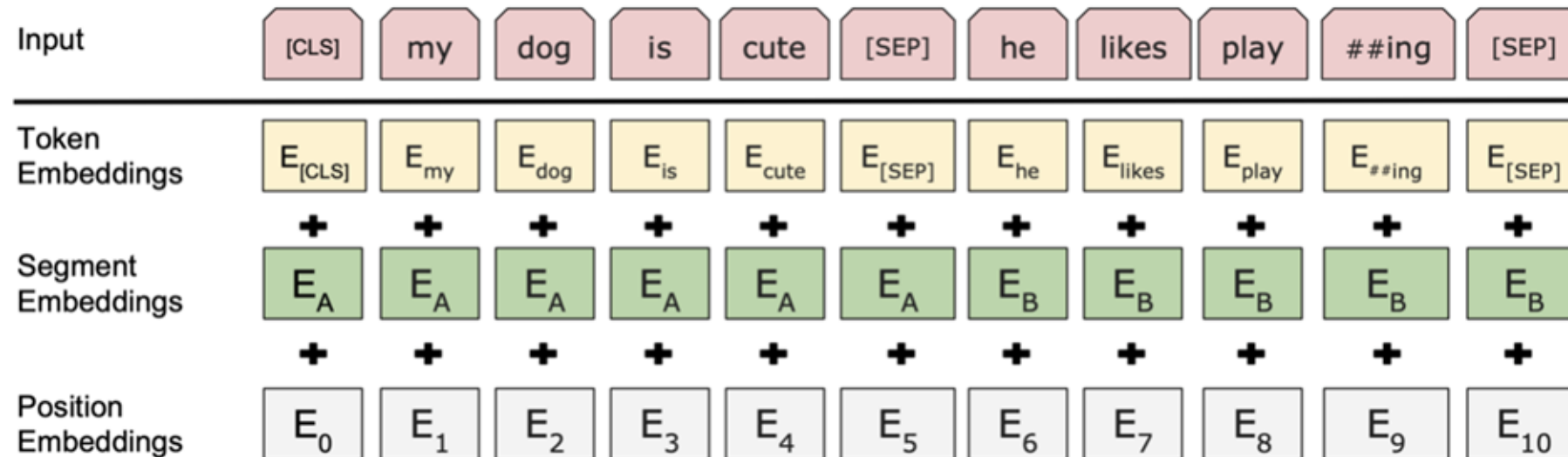


BERT Features

- Training corpus was comprised of two entries
 - [Toronto Book Corpus](#) (800M words), and
 - English Wikipedia (2,500M words)
- Two versions of BERT (L stands for the number of layers, H stands for the hidden size, A stands for the number of self-attention heads)
 - BERT-Base: $L = 12$, $H = 768$, $A = 12$, Total parameters = 110M
 - BERT-Large: $L = 24$, $H = 1024$, $A = 16$, Total parameters = 340M



Input Representation of BERT



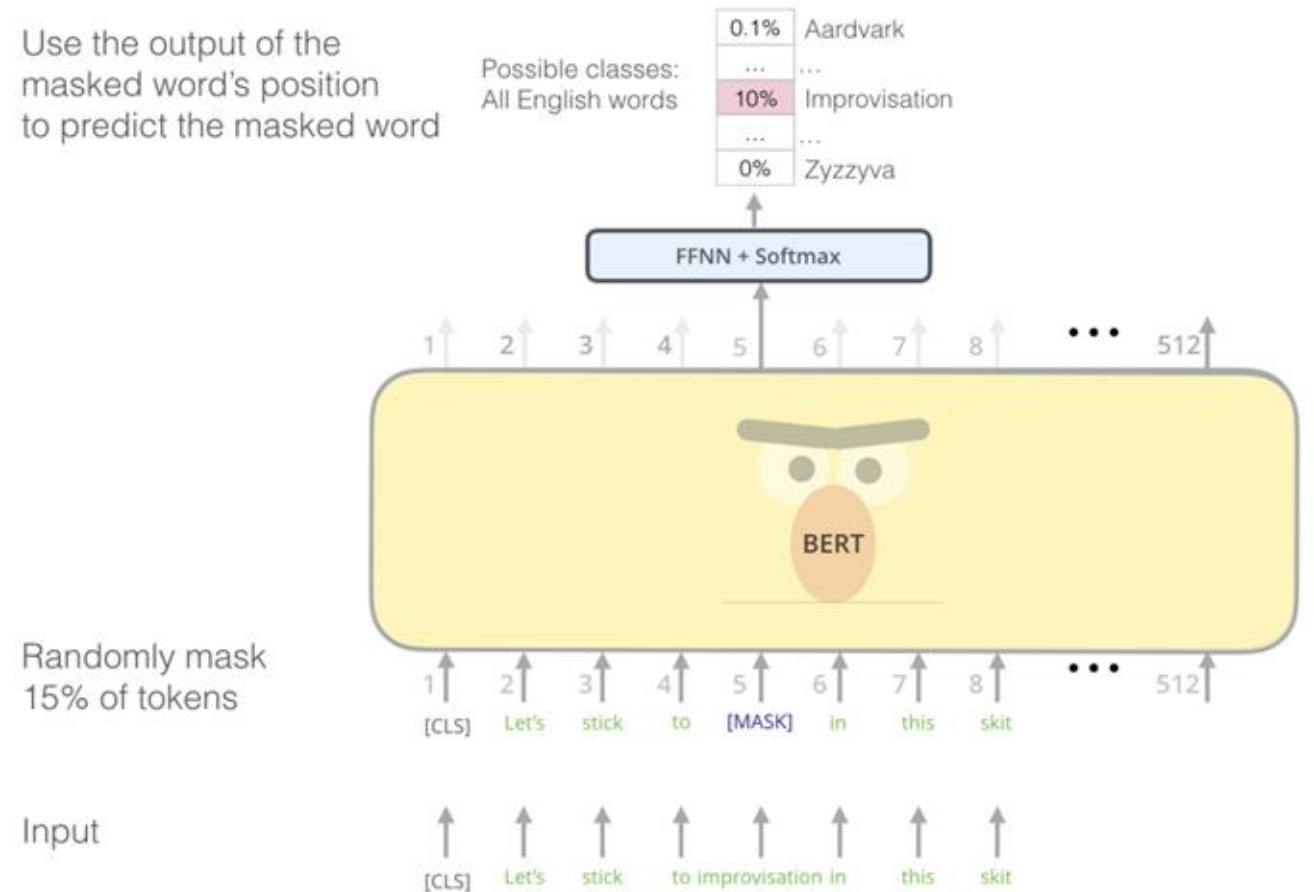


BERT Training – Predicting word at a position

- Masked language model (Masked LM)

That's [mask] she [mask] -> That's what she said

- Randomly masks 15% of tokens in the input and model is trained to predict the masked word



BERT's clever language modeling task masks 15% of words in the input and asks the model to predict the missing word.



BERT Training – Two sentence tasks

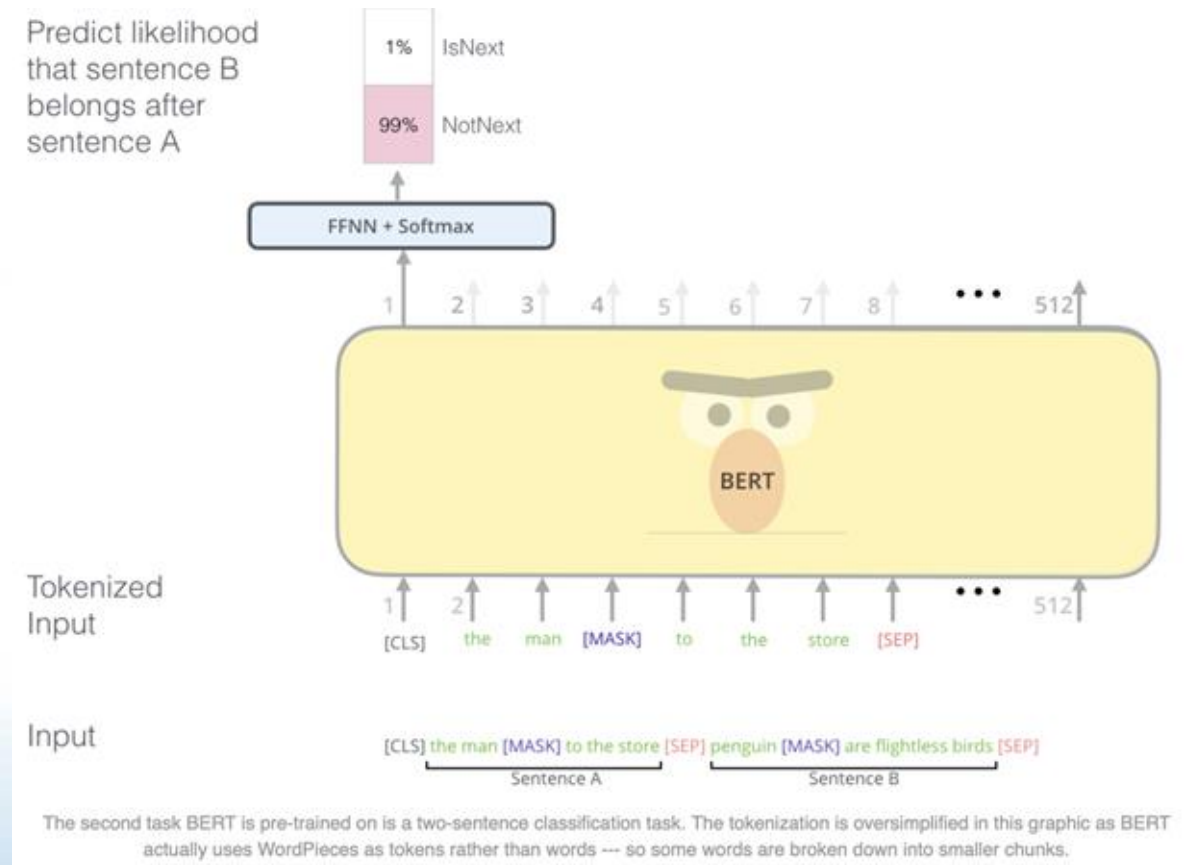
- Next sentence prediction (NSP)
- Task: Given two sentences (A and B), is B likely to be the sentence that follows A, or not? (binary classification)

Input = [CLS] That's [mask] she [mask] . [SEP] Hahaha, nice! [SEP]

Label = IsNext

Input = [CLS] That's [mask] she [mask] . [SEP] Dwight, you ignorant [mask] ! [SEP]

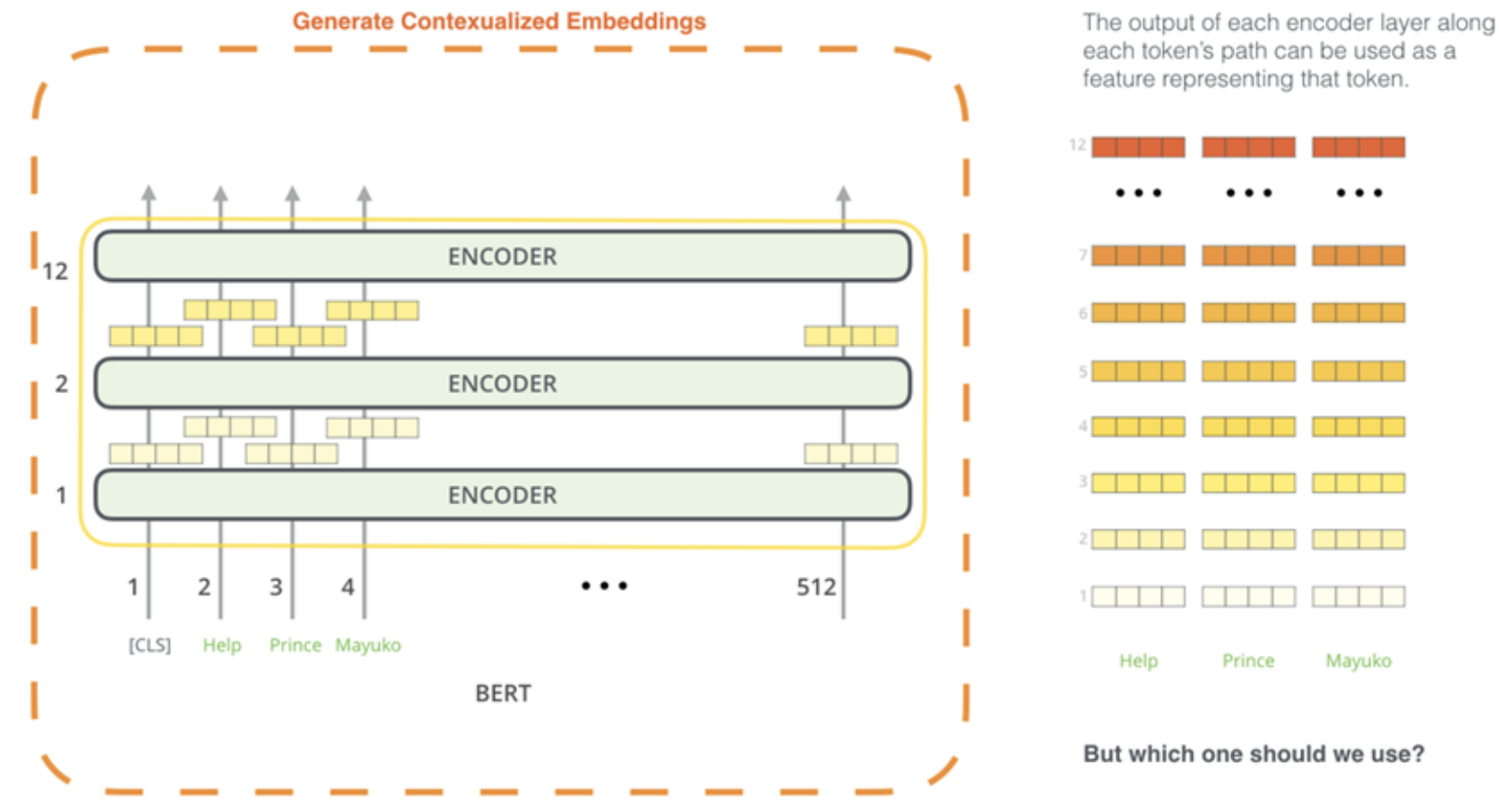
Label = NotNext





BERT for Feature Extraction

- Using BERT to create contextualized word embeddings



- 12 embedding vectors for each word from the output of 12 encoder layers
- Which embedding(s) to use for a given task?



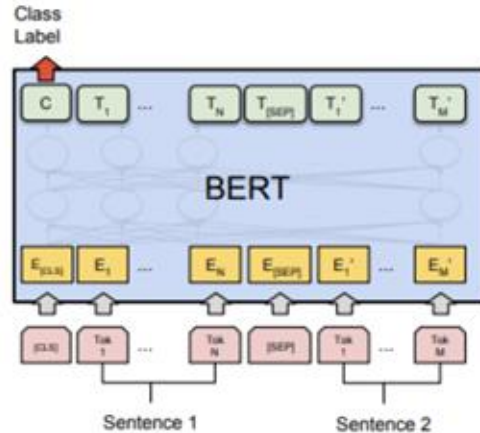
Different ways to create contextualized embeddings for a word using BERT

What is the best contextualized embedding for “Help” in that context?
For named-entity recognition task CoNLL-2003 NER

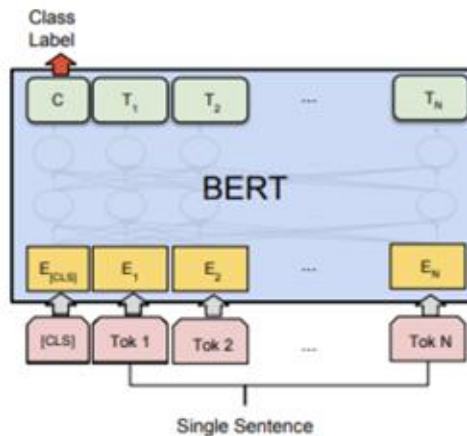
		Dev F1 Score
2		
...		
7		
6		
5		
4		
3		
2		
1		
	Help	
First Layer	Embedding	91.0
Last Hidden Layer	12	94.9
Sum All 12 Layers		95.5
Second-to-Last Hidden Layer	11	95.6
Sum Last Four Hidden		95.9
Concat Last Four Hidden		96.1



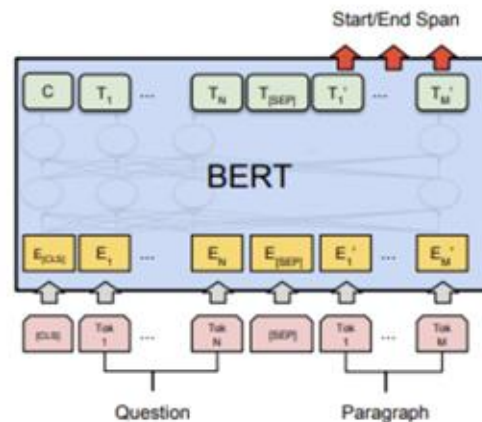
Using BERT for different Downstream Task



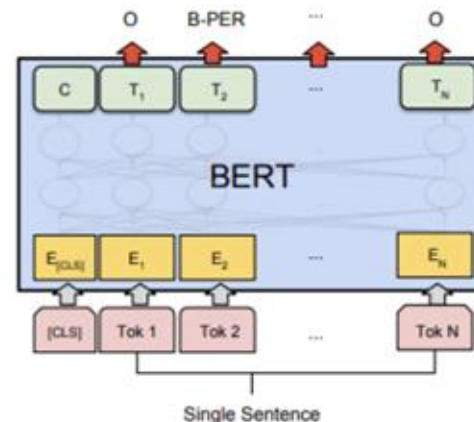
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER



Fine-tuning

Fine-tuning

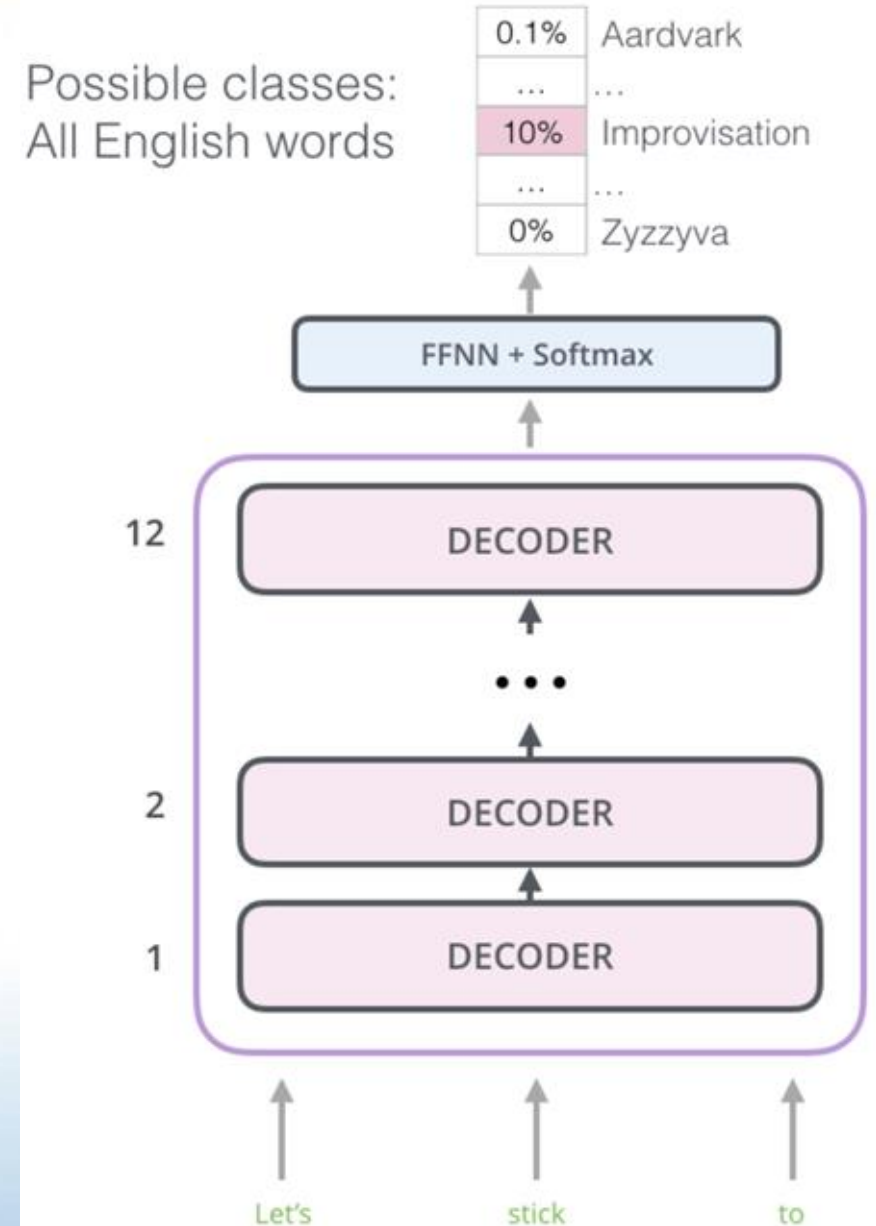
The model is trained via repeated gradient updates using a large corpus of example tasks.





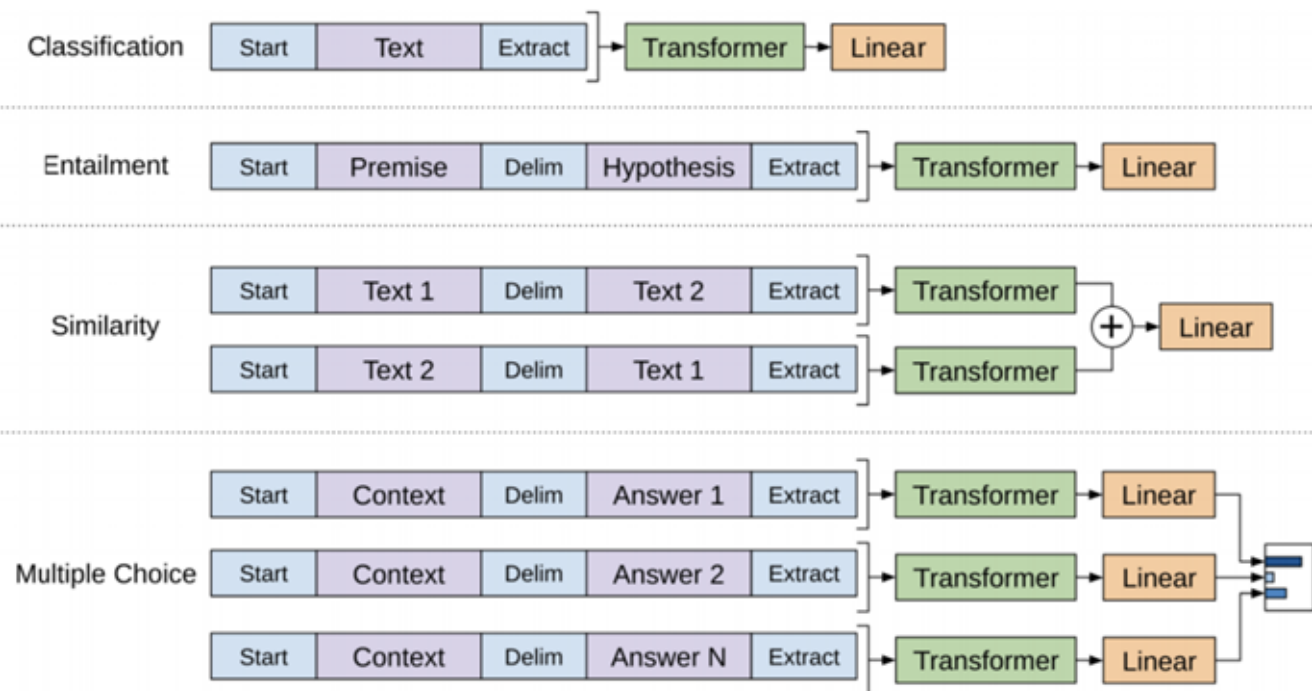
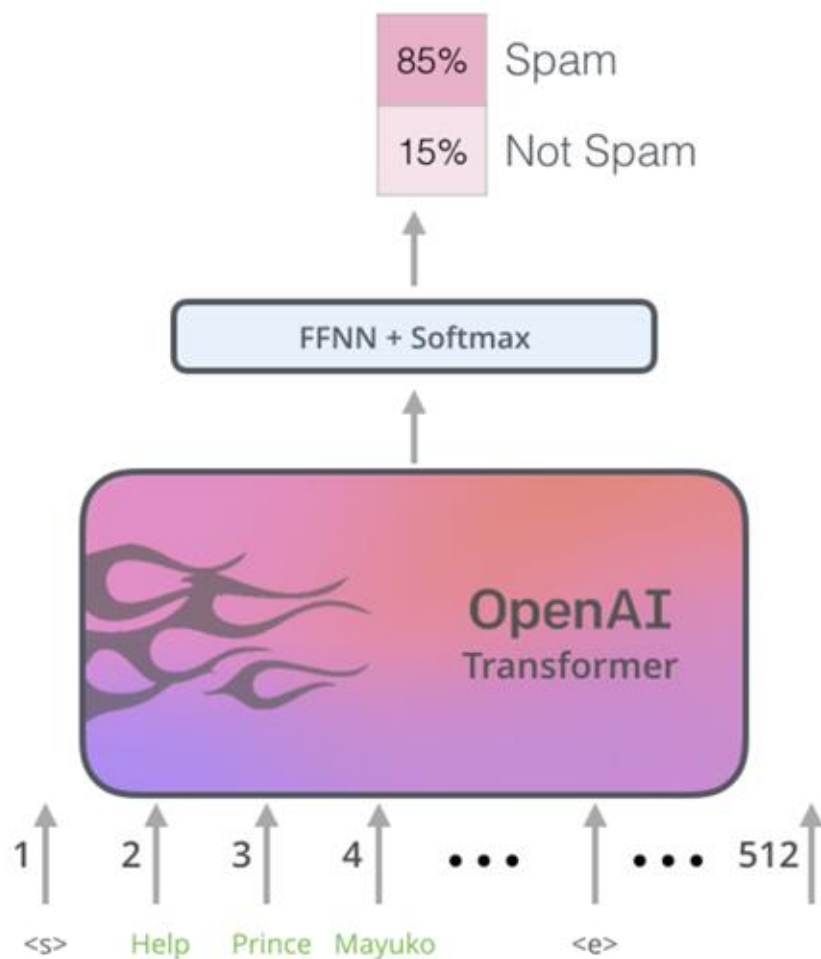
OpenAI Transformer

- Pre-training a transformer decoder for language modeling
- Decoder masks future tokens so can be trained on next word prediction task
- Model has 12 decoder layers stacked
- Only masked self attention layers





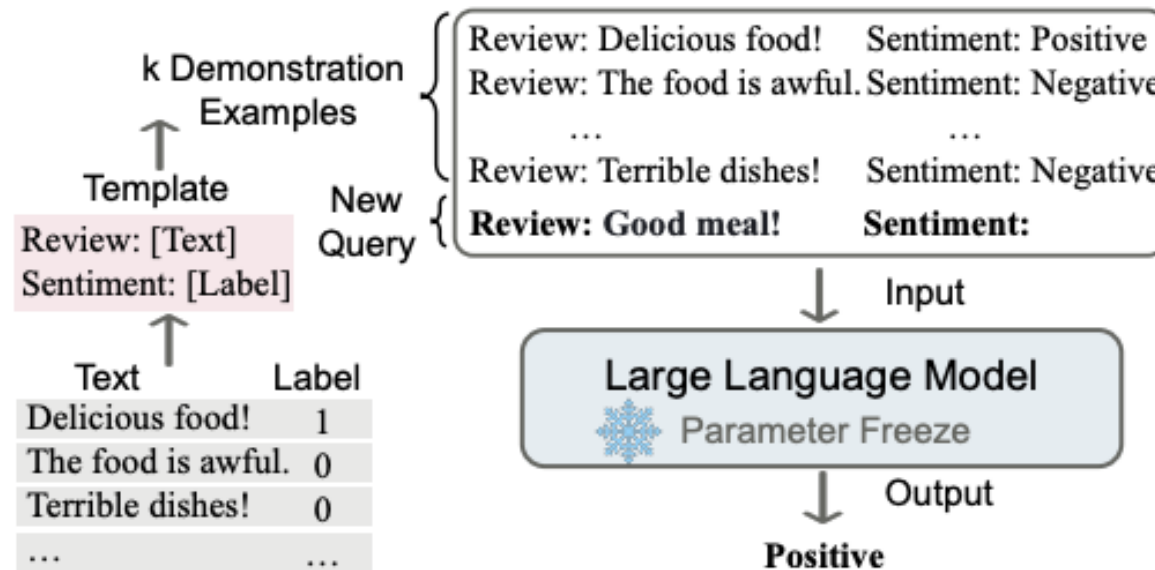
Transfer Learning to Downstream Tasks





In-context learning (ICL)

- Learning from few-examples in the context (input to the LLM)
- Does not require weight updates as in fine-tuning
- Learning from analogy



The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French:  ← task description
2 cheese => .....           ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French:  ← task description
2 sea otter => loutre de mer    ← example
3 cheese => .....             ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French:  ← task description
2 sea otter => loutre de mer    ← examples
3 peppermint => menthe poivrée ← examples
4 plush girafe => girafe peluche ← examples
5 cheese => .....             ← prompt
```



Few Shot (FS) Learning

- The model is given description of the task and a few demonstrations of the task at inference time as conditioning but no weight updates are allowed
- Gives K examples of context and completion, and then one final example of context, with the model expected to provide the completion

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

1	Translate English to French:	← task description
2	sea otter => loutre de mer	← examples
3	peppermint => menthe poivrée	←
4	plush girafe => girafe peluche	←
5	cheese =>	← prompt



One Shot (1S) Learning

- Only one demonstration is allowed, in addition to a natural language description of the task.
- Most closely matches the way in which some tasks are communicated to humans.
- With one example sometimes it is easier to communicate the content or format of a task vs if no examples are given.

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

1	Translate English to French:	← task description
2	sea otter => loutre de mer	← example
3	cheese =>	← prompt

.....



Zero Shot (ZS) Learning

- No demonstrations are allowed, and the model is only given a natural language instruction describing the task
- Some settings zero-shot is closest to how humans perform tasks
- Can be “unfairly hard” in some cases

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

1	Translate English to French:	← task description
2	cheese =>	← prompt

- From OpenAI
- Decode only architecture

GPT-3 family of models

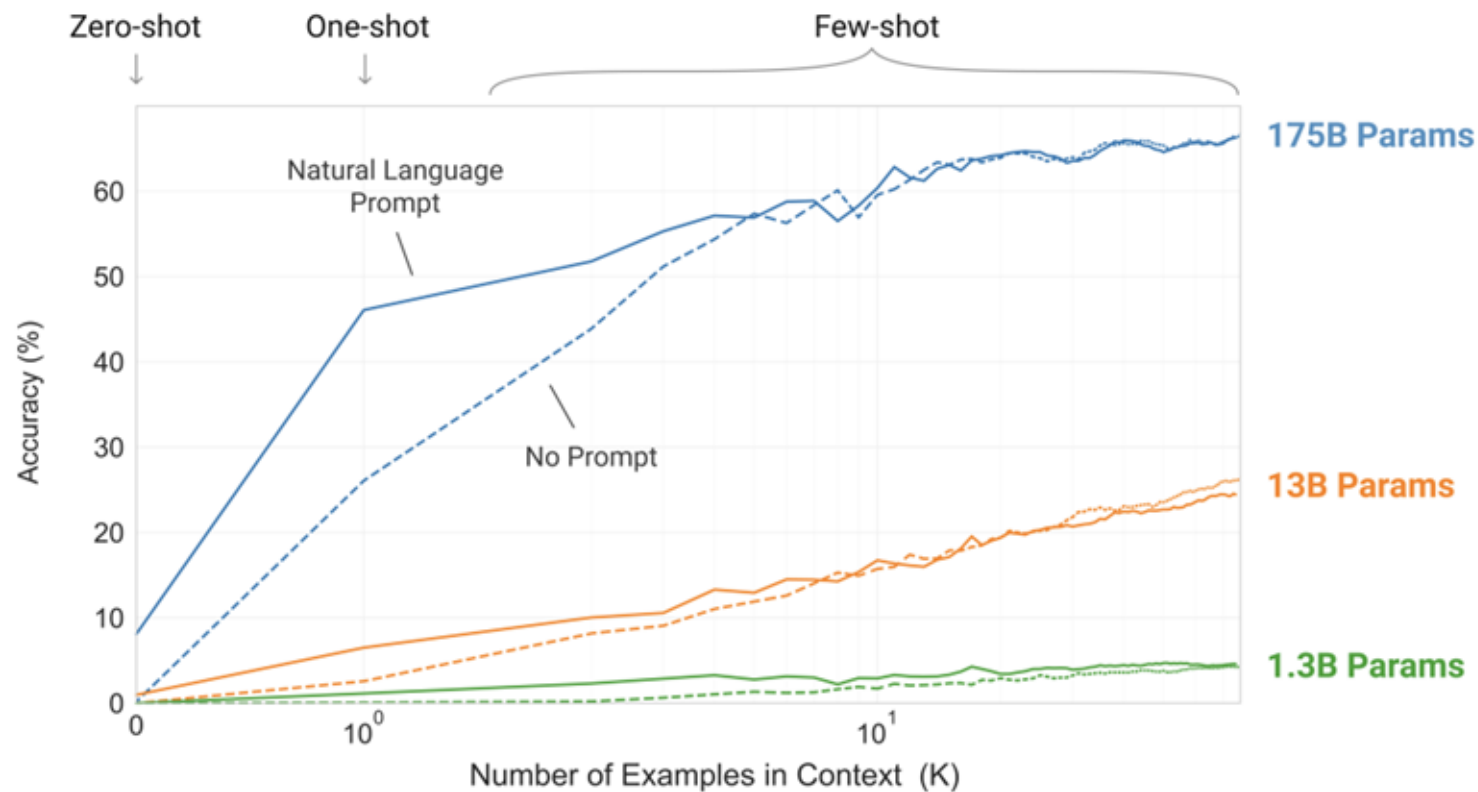
Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

GPT-3 training dataset

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4



GPT-3 performance with ICL



- Task: remove random symbols from a word, both with and without a natural language task description
- Steeper “in-context learning curves” for large models demonstrate improved ability to learn a task from contextual information

Larger models make increasingly efficient use of in-context information.



LLaMa

- From MetaAI
- Decode only architecture

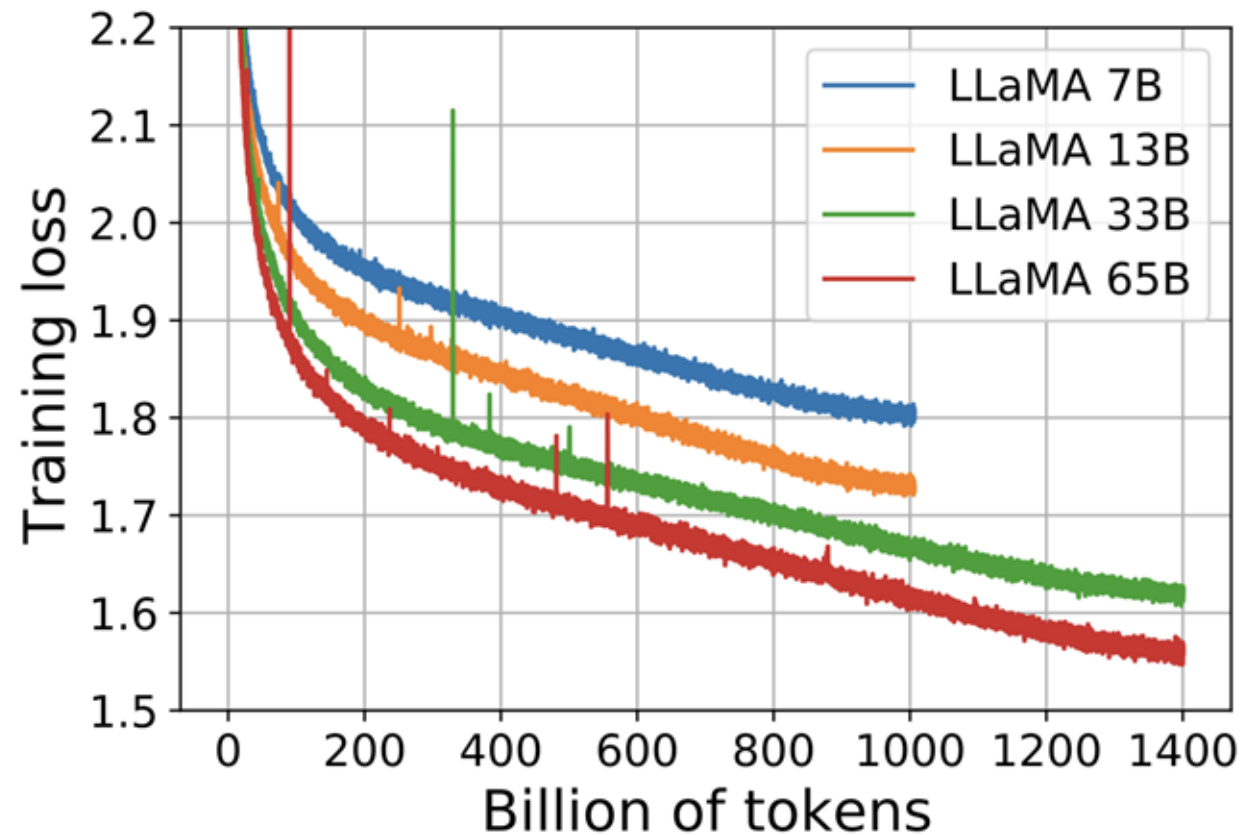
LLaMa family of models

params	dimension	n heads	n layers	learning rate	batch size	n tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

LLaMa training dataset	Dataset	Sampling prop.	Epochs	Disk size
	CommonCrawl	67.0%	1.10	3.3 TB
	C4	15.0%	1.06	783 GB
	Github	4.5%	0.64	328 GB
	Wikipedia	4.5%	2.45	83 GB
	Books	4.5%	2.23	85 GB
	ArXiv	2.5%	1.06	92 GB
	StackExchange	2.0%	1.03	78 GB



LLaMa Training



LLaMA-33B and LLaMA- 65B were trained on 1.4T tokens. The smaller models were trained on 1.0T tokens. All models are trained with a batch size of 4M tokens.